

Crypter

6142 / 33611

Saul Needham

Contents

Contents.....	2
Analysis	4
The problem.....	4
Time management.....	4
Data collection	5
Existing programs.....	9
Encryption / Decryption methods.....	10
Hardware / Software	11
Design.....	13
Pre-planning.....	13
Server connection	14
Log in screen	15
Register screen.....	16
Notepad UI.....	18
Chatroom UI.....	20
Encryption / Decryption.....	21
Variable list	23
Test plan.....	24
Development.....	26
Setting up the server.....	26
Setting up the client.....	28
Account creation GUI.....	29
Login page	29
Register page.....	31
Communicating user details	32
Setting up files	40
Setting up keys.....	42
Main GUI (server side)	45
Main GUI (client side)	49
Final tweaks	57
Program logo.....	57
Resizing Tkinter window	59

Line breaks in global chat.....	60
Redirecting to login page	61
Printing messages on GUI	62
Testing.....	65
Checking test plan.....	65
User login	65
User registration	67
Cyphering messages.....	70
Global chat	73
Extra testing	74
Interaction with global chat.....	74
Global chat updating.....	75
Updating of usernames list	78
Fixing empty passwords.....	79
Handling clients leaving	79
Evaluation	83
Requirements.....	83
Client usability.....	86
Client feedback	88
Overview	89
Code	90
Server code	90
Client code	99
References	111

Analysis

The problem

The client's problem

My main client is Joel Mathews, who wants a program which allows people to send encoded messages to one another, which cannot be intercepted and read by the wrong recipient. He has also said a login system in which people will have their own accounts and their own set of keys to send messages securely.

The idea

The plan for the project is to create an online encryption program, in which people can register, log in, and encrypt/decrypt messages to then send to other users. This would use asymmetric encryption meaning each user would have their own private and public key, to make the messages as secure as possible.

Stakeholders

I believe that this program could also be adapted easily for many other people, such as businesses who want to keep their data and messages secure. Teams of people who need to share sensitive information would benefit from this program as they know they can securely send messages to other people in their team or workplace without worrying about anyone else intercepting or reading what they're saying to one another.

Time management

To plan out how I will spend my time, I will use a Gantt chart to display how I will divide it up. This is important so I make sure I do not spend too long on one section, and not leave enough time for a different, possibly more important one. Throughout the time of making my program I will be documenting the parts of the project I am working on.

Gantt Chart

Task	30/05	10/06	21/06	08/07	06/09	30/09	16/12	28/02	24/03
Problem Definition									
Investigation									
Analysis									
Design									
Development									
Testing									
Documentation									
Evaluation									

Data collection

For the main part of data collection, I need to decide on what methods are going to be best for me to decide what to add and abstract from the program. I have chosen to conduct both a survey first to get a wide range of data and secondly, I will conduct an interview with Joel to get the more specific details of the project.

Survey with wider clientele

Here I conducted a survey with some of the people who Joel said he would like to use the program in the future. I chose to use this as one of my data collection methods as, as I stated, it helps get a wider consensus on what the program should entail. Hopefully I should also get some suggestions of what could be added and get some clear one sided opinions on what to do.

The Survey

Questions	Responses
	15

Crypter survey

Essentially, this is an online, cross network program in which a user will be able to encrypt and decrypt message to and from users, using public and private keys. Pls answer the questions on it, cheers x

1. Would you want a chat function, where users could send messages in the running program itself? *

☐ Yes

☐ No

2. Should limitations be set for usernames and passwords (e.g. must be over a certain length, must contain capitals etc) *

☐ Yes

☐ No

3. How important would a feature be where you could view all user public keys, without them needing to send it to you (1 - not important, 5 - important) *

1

2

3

4

5

4. Rank these in order of what you believe to be most important (top) to least important (bottom) features *

Hashing system for login

Chat room

Profile customization

Screen customization (e.g. dark mode)

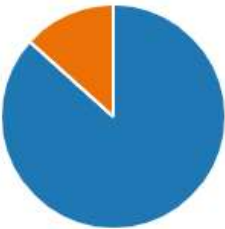
5. Any other suggestions?

Enter your answer

The Results

1. Would you want a chat function, where users could send messages in the running program itself?

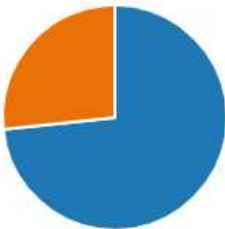
[More Details](#)



2. Should limitations be set for usernames and passwords (e.g. must be over a certain length, must contain capitals etc)

[More Details](#)

[Insights](#)



3. How important would a feature be where you could view all user public keys, without them needing to send it to you (1 - not important, 5 - important)

[More Details](#) Insights

15
Responses

3.40
Average Number

Rank these in order of what you believe to be most important (top) to least important (bottom) features *

Hashing system for login

Chat room

Profile customization

Screen customization (e.g. dark mode)

Out of 15 responses, here was how the following features were ranked in order with assigned points:

Hashing system for login – 47

Chat room – 38

Screen customization – 37

Profile customization – 28

Survey analysis

As shown by the results, people believed hashing the logins was the most important feature of the 4 mentioned, followed by a chat feature and a visual customisation choice. The majority also said in a separate question that the chatting function would definitely be one they wanted to see so this will be a fairly high priority for added features. People also said limitations should be set for passwords so I should try adding that in as well.

Client Interview

Here is a transcript of the interview that was conducted with the main client, Joel Mathews, in which I asked important questions to understand the project better:

Me:

"Hello Joel."

Joel:

"Hello Saul."

Me:

"As you know, I sent out a survey to some of the clients you said you would like to eventually use the program and I would like to discuss the results from that, as well as ask you some other questions about the program that I have."

Joel:

"Okay."

Me:

"So firstly, I have shown you the results from the survey, what do you think about the answers?"

Joel:

"I agree with most of the results, however, I feel like some of the options were more necessary features rather than optional ones, such as the Chat room one. I feel like security is a really important factor, and therefore the hashing system is also incredibly necessary."

Me:

"I see, so both the chat and the hashing logins will be top priority once the interface and encryption are set up, do you have any other suggestions that you would possibly like to see on the program?"

Joel:

"Not really, I think adding anything else would ruin the simplicity of the interface and would just be unnecessary. I also like the idea of it just being a monotone, plain colour scheme so it is easier on the eyes and doesn't take away from the program itself."

Me:

"Okay, that can be done. Should that be the case for just the login and register pages or the program as a whole?"

Joel:

"I like the idea of the program as a whole being like that."

Me:

"Thanks a lot Joel, I think that's enough for me to go off."

Joel:

"No problem fella"

Interview analysis

The interview with Joel gave me a good overall illustration on both how he wanted the UI to look and also how he would like it to run. One key point about the aesthetics he stated was that he wanted a fairly plain, non-distracting colour scheme for the whole program which I will likely make possible using Tkinter. He also spoke about how hashing the login system would be a good idea, which was also made clear in the survey so implementing those will be important.

Program requirements

Necessary

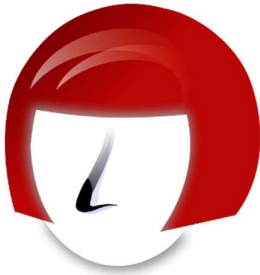
- Online server, for users to connect to
- A functioning login system
- A functioning registering system
- Encryption and decryption algorithm
- Key Creation system
- Clients preferred UI

Optional

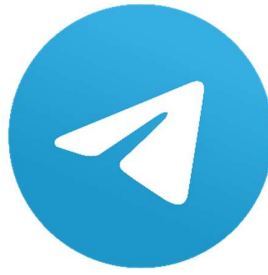
- A hashing system for user logins, mainly for passwords
- Extra details for profiles
- Be able to select all users public keys from one account
- A chat in which users can publicly send messages

Existing programs

Many apps that are used almost every day by people use asymmetric encryption, as it is a lot more secure than symmetric. Some great examples could be WhatsApp and Telegram, as every message sent through them from one person to another is encrypted when first sent and the decrypted when received by the second person.



Kleopatra



Telegram



WhatsApp

An example of a similar existing program could be the application 'Kleopatra'. In this you can also create an account and you are given both a private and public PGP (Pretty good privacy) key, to then send encrypted messages with. Kleopatra is widely used for sending messages, especially on The Onion Router (TOR), when people want to privately speak to one another with the asymmetric encrypting making it very hard for the wrong person to intercept and read their message.

Summarisation

In conclusion, I would like to include or at least simulate the PGP encryption methods used in Kleopatra, as I believe it's the best and most secure encryption method to use. I will also make the program use asymmetric encryption (using two keys) as all of these existing applications use it due it being more secure than symmetric.

Encryption / Decryption methods

Caesar cipher

Likely the most well-known encryption method is the Caesar cipher, a very simple way of disguising text by shifting characters a certain amount so that the text is completely jumbled. For example, if we shift along the characters of the alphabet using the word 'taxi' using the key of 6 characters, we would end up with the encrypted word 'zgdo'. This would therefore be outputted as 'zgdo,6', including the key so that it can be reverse decrypted. This on its own seems like a very simple method, and fairly easy to decrypt. However if we improve the key by making it much larger, we can assign each number of the key to each character in the message and make it much harder to crack – especially if the second key (private) responsible for decrypting is not known by users.

Vernam cipher

In the Vernam cipher, the message and key both have to be phrases, so they can be converted from ASCII into their respective binary values. These numbers are then masked using an XOR gate, and the result of the mask is the encrypted message. The main issue with this method is that both the message and the key have to be the same length, and therefore this would fix how long the users message would have to be in order to be encrypted, which I would not want to force clients to do. For this reason, I will likely not use this method.

Character insertion

One other method I looked at using keys was the use of character insertion. This is where using the numbers in a key, a respective to key number of random characters are inserted into the message. For example, again using the word 'taxi' and using the key '214291424978242359238535', this word could be encrypted into 'tk@aJxO(7)iBc'. Due to the message only having 4 characters, only the first 5 characters of the key would be used to insert characters into the message.

RSA encryption

The final method I looked at, and likely the most complicated was RSA encryption. This method works by having a public key pair of numbers (e.g. 5 and 14), and then a user's text message which would later be converted to ascii. The ascii of the messages character (e.g. if the corresponding number was 2) is then raised to the power of the first key number – so 2^5 . The result of this is then modulus by the second key number so the result would be $(2^5) \bmod 14$, which is $(32) \bmod 14$, in turn giving us a result of 4. The user then receiving the message of 4 would then have a working private key pair which would be able to decrypt the public key. In the case of public(5,14), a working key would be private(11,14). Now when the receiving user does the same method with the outputted 4 using their key pair, they get $(4^{11}) \bmod 14$. This finally now returns the original value of 2, and gives the user the original character. Although this method works, the initial key creation is the real complicated part of the process, and therefore generating a key of large size which would work on long messages would be far too time consuming for the server to work out, especially with multiple clients encrypting messages concurrently.

Hardware / Software

Why is this problem solvable by computational approach?

This problem arguably can only be solved by computational methods, or is at the least made incredibly simpler and more efficient when done so:

- **Login system.** The login part of the program will be heavily aided by a computer as the computer can store all the logins and passwords. Not only this but the hashing of the logins will be done by the computer to make the server and clients more secure. Finally, the front-end interface will be supported by the computer and more specifically Tkinter, a python module.
- **Key creation.** During key creation, a user's public key and private key are both created, in a way that they are linked, but cannot be worked out from only having one of the keys. This can only really be done computationally as key creation is an incredibly complicated and secret processes.
- **Encryption.** One key creation is done, the program will have to use peoples public and maybe private keys to encrypt a message for people to send to others. Complex algorithms will be used in order to hide the contents of the message for the intended recipient, so a computer is heavily necessary for this.

- **Decryption.** Like encryption, the decryption stage will also use algorithms in order to decrypt the message back to its original text.
- **Online Feature.** As the whole program will be online on which users are connected to a server to communicate, I will use computers to run client and server (front end and back end) as they will be able to manage and communicate between all users on the server.

Limitations

Before starting the project, I must look over and assess the limitations I may come across. One of these could be issues linked to clients and connecting to the server. Due to limited hardware, there may only be a set number of clients that are able to connect at once, which shouldn't be too much of a problem, but it is one that I must keep in mind.

Not only this, but there is a possibility that the time set might not be long enough for me to add in all the features that I want to. Therefore, time management will be an important part of the project, so I know whether I am meeting certain deadlines for different parts of the program.

One more limitation to my program could be the fact that in order for people to connect online, I would need to have the server constantly running in order to manage all online traffic. This could be an issue because as stated before, the hardware on which the server program will be running is limited in performance, possibly slowing down the main program.

The programming language

For this project, I have chosen to use python to write the main program on. This is because I am familiar with python and would also like to expand my knowledge on it. Python also contains the modules I would like to use (such as Tkinter and Hashlib) which makes it ideal for me. Predominantly I will be using Visual Studio Code as my IDE as I have used it for many years, and I also like the debugging and error detection software it has.

The hardware

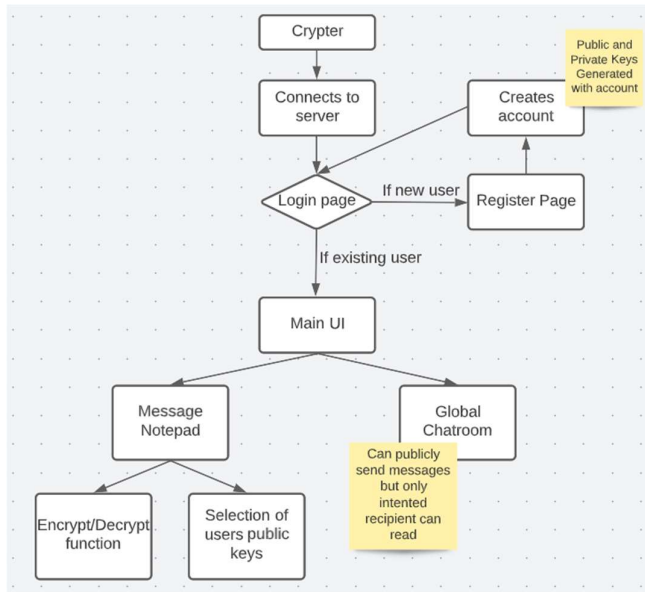
The processor which the computer I am running the server on will be an Intel Core i5. The processor uses 4GB of RAM, 6GB of cache and has 4 cores. This will be very useful having multiple cores as the module I am planning on using called "Threading" uses parallelism in order to allow the computer to multi-task on the server end.

Design

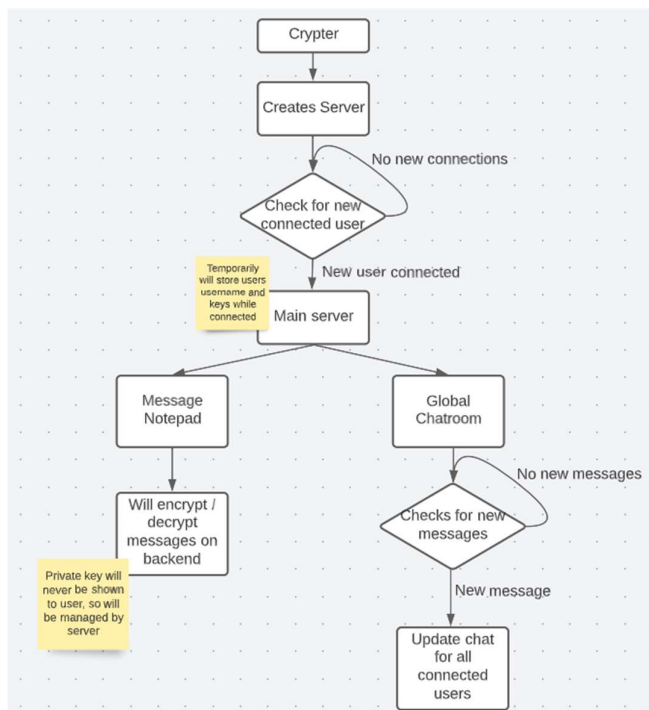
Pre-planning

As I will have two sets of code, one for the server and one for the client, I will need to plan out how both of these will roughly work. I am planning on breaking down my design into 4 key parts – Log in page, Register page, Message notepad, and the Online chatroom. I will also need a part at the start of the program which will connect clients to the server.

Client Plan



Server Plan



Server connection

Design

For this, I will need to separate pieces of code which work with one another for the client and for the server. The server code will of course first set up the server and immediately start waiting for clients. The client code on the other hand will attempt to join the server and then will be sent to the login UI if it connects successfully.

Server connection

```
Set port
IPadress = current IPadress
Start server(port, IPadress)
SERVER = RUNNING
WHILE SERVER RUNNING:
    WAIT until client connection:
        Add client to clientLIST
        WAIT until clientLIST[client] login :
            Add login info to usernamesLIST
            Add login info to privateLIST
            Add login info to publicLIST
```

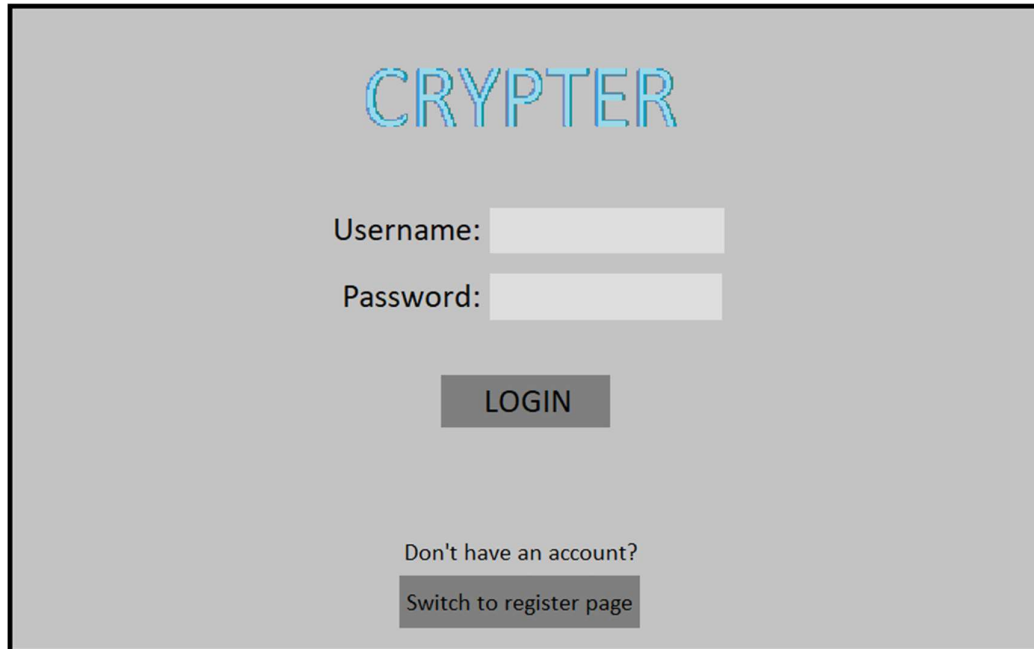
Client connection

```
Set port
Import IPadress
Connect server(port, IPadress)
IF connection = succesful:
    Open Login page
    #links to Login UI
ELSE:
    Output "connection error"
    QUIT
```

Log in screen

Design

As my client stated in the interview, they wanted a simplistic design across the whole program. For this reason, I will only add the necessary parts to the pages. The log in screen specifically will consist of a **username box**, a **password box**, a **login button** and a **switch to register page button**, all which can be added using Tkinter. I will also try to keep to the requested simple colour scheme.



Username box

For the username box upon logging in, the entry will be checked against a file containing all usernames of existing users. If the username is within the file, the place of it will be saved as an index variable.

Password box

The password box will work similarly in that it will also have a file of passwords affiliated with accounts. The program will check the index of the password in the list with the index of the entered username and check if they are the same. Not only this, but upon entry of the password, I am going to attempt to add a hidden function where the characters of the password are replaced with stars (*). Finally, I may also add in a hashing system so the passwords are never actually stored, but rather just the value of them.

Login button

When the login button is pressed, this will run the function which will check the username against the password and see if they match in the files to an account. If they do, then this will log into the main program.

Switch page button

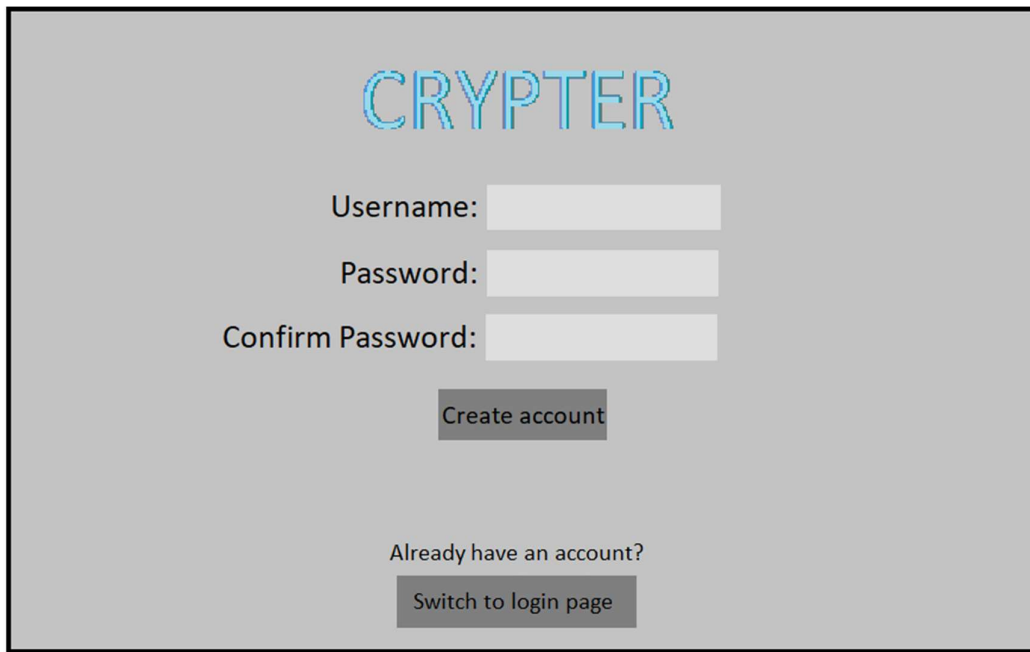
The switch page button will work by creating a new canvas which will contain all the parts needed for the register page, This will destroy the login canvas making it appear to the user that they have switched page.

```
username = username input
password = HASHED password input
WAIT until button pressed:
    IF login button:
        IF username is in username file:
            IF index of username in file is index of password:
                Return "Log in Successful"
            ELSE:
                Return "Password Incorrect"
        ELSE:
            Return "Account not registered!"
    ELSE IF switch page button:
        Switch to Register page
```

Register screen

Design

Similarly to the log in page, the register screen will contain a **username box**, a **password box**, a **confirm password box**, a **create account button** and a **switch to login page button**. I will again keep to the requested simple colour scheme also.

A registration form titled "CRYPTER" in a blue, pixelated font. The form is set against a light gray background with a black border. It contains three input fields: "Username:", "Password:", and "Confirm Password:", each followed by a light gray rectangular box. Below these fields is a dark gray button labeled "Create account". At the bottom, there is a link "Already have an account?" followed by a dark gray button labeled "Switch to login page".

CRYPTER

Username:

Password:

Confirm Password:

Create account

Already have an account?

Switch to login page

Username box

When a username is entered to register, it will be checked that it is not already taken in the username file, which will always be managed by the server side computer. If the username is not taken, it will be accepted by the program and added to the file.

Password box

Contrarily, when registering an account, the program will not have to check the existing passwords file as it does not matter if two accounts share the same passwords. The only thing the password will be checked against is the confirm password entry. When passwords match up and an account is created successfully, I will make it so the passwords are hashed rather than stored as they are in the file, so they can be checked when a user logs in. This also adds security if the files were ever breached.

Confirm password box

As mentioned in the last section, the confirm password box will simply check if the two passwords match up when an account is being created.

Create account button

When pressed, the username and passwords will be checked and added to the files. Once the account is created, the user will be redirected back to the login page by running the switch page function.

Switch page button

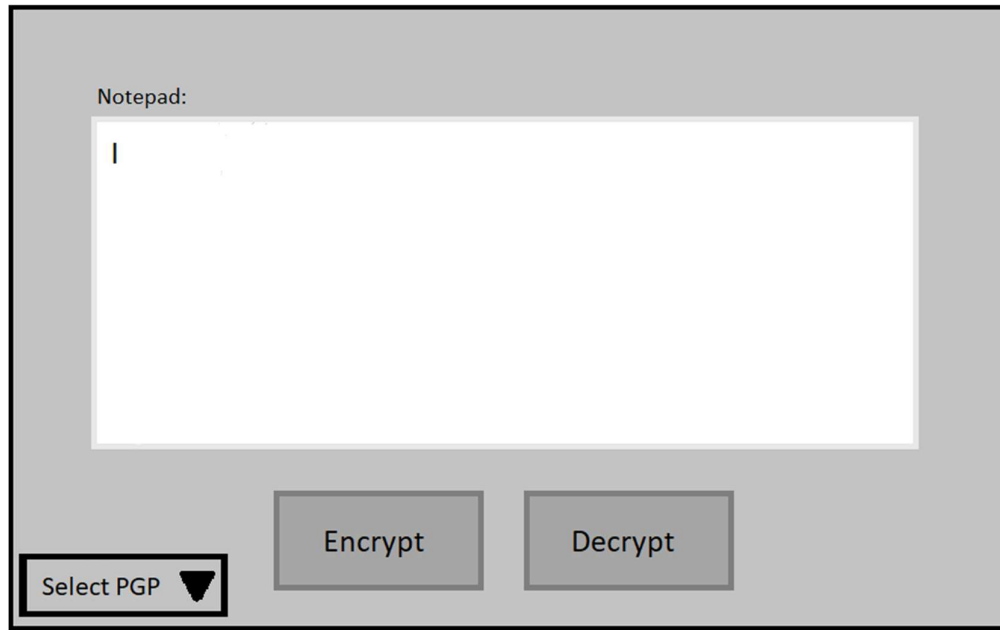
This button will work in practically the same way as on the log in page, it will create a new canvas again with all of the pieces for the log in screen, and then will delete the current register canvas.

```
username = username input
password1 = password input
password2 = confirm password input
WAIT until button pressed:
    IF login button:
        IF username is in username file:
            Return "Account username taken!"
        ELSE:
            IF password1 is same as password2:
                HASH password1
                Add password1 to password file
                Add username to username file
                Return "Account created successfully!"
            ELSE:
                Return "Passwords don't match!"
    ELSE IF switch page button:
        Switch to Login page
```

Notepad UI

Design

Once the user has logged in to their account, they will be brought into the main program which will be split into two main sections, one of which is the notepad. Here the user will be able to select a public key from the **key menu**, type into the **notepad** where they can then **encrypt** or **decrypt** a message using the two buttons.



PGP key menu

In the PGP menu, the user will be able to choose from a list of existing users who all have accounts on Crypter. Once one is selected, the program will copy their public key for the user and will therefore allow them to encrypt a message only readable by the intended person with their private key.

Notepad

This section will just be a text entry box in which the user can enter either a message they are wanting to send to someone or an already encrypted message sent to them by someone else. I will try make it so the user can add any characters they want and where the encryption still works well.

Encrypt button

When pressed, the program will take the public key of the user selected in the key menu and will encrypt the message in the notepad accordingly. This message will then be ready to send off to the recipient.

Decrypt button

When pressed, the program will use the users private key (only stored locally by the server computer) by requesting it and using it temporarily. By using the private key, this will decrypt a message written by someone else who has used their public key to initially encrypt the message.

```

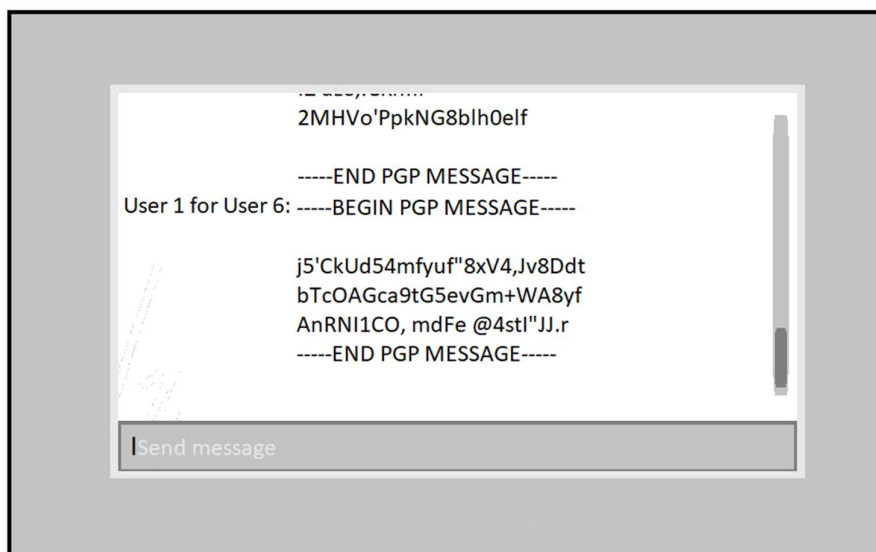
text = notepad input
WAIT until button pressed:
    IF encrypt button:
        IF username is in username file:
            Return "Account username taken!"
        ELSE:
            IF password1 is same as password2:
                HASH password1
                Add password1 to password file
                Add username to username file
                Return "Account created successfully!"
            ELSE:
                Return "Passwords don't match!"
    ELSE IF decrypt button:
        Switch to Login page

```

Chatroom UI

Design

The final part of the program will be the chatroom section. In here, users will be able to share their encrypted message with one another. This will likely be on the same screen as the notepad to make it easier to use the whole program but I may split them into separate sections. Also, as shown in the sketch below, I will try to make it so users can state which users their message is intended for. This will probably be a menu in which the client can select another.



Message box

This simply will be an Tkinter entry box, in which users will enter their message to be shared on the global chat for all clients. Users will be able to then copy the message and decrypt it in their notepad if the message was intended for them.

For user menu

The user should also be able to have some way of indicating what user the message is meant for. This will probably be a menu in which the user can select a connected client and the message will be highlighted for them on the chat so it is clear that it is for them.

WHILE SERVER RUNNING:

```

    WAIT until message received from client:
        IF message text empty:
            Return "Cannot send empty message!"
        ELSE:
            Global Return message

```

Encryption / Decryption

Method

During research in my analysis stage, I looked at multiple encryption and cyphering methods that I could possibly use in my program. Keeping in mind what my client wanted the encryption to simulate (they mentioned they wanted the program to be similar to an existing application called 'Kleopatra'), I believe that the best method to give a similar appearance and secure procedure would be to use character insertion, dependant on a user's key. To add to the security of the encryption, I also could possibly add in a shifting cipher similar to that of the Caesar cypher so that none of the users original message is contained in the encrypted version.

Encryption

As the inserted characters will be removed upon decryption, only the actual message content will need to be shifted. Because of this, we will do the shift cipher before the random character insertion.

```

shift = 'abcdefghijklmnopqrstuvwxyz'
randomChars = '1234567890qwertyuiopasdfghjklzxcvbnm,<.>;:@[{}]#~/?'
publicKey =
'4122746237526310935238753252345234062346365463456340956345634563456
3456345634570634563456345345234623570250235915634563455235'
encryptedMessage = ''

```

```

message = input('Enter message to encrypt')
#caeser cipher
for character in message:
    for number in key:
        find character index in shift
        character = shift[index+number]
#character insertion
for character in message:
    insert = ''
    for number in key:
        insert = insert + randomChars[random]
    encryptedMessage = encryptedMessage + character + insert
output(encryptedMessage)

```

Decryption

We will now reverse this process for decryption, using the opposite key in the pair.

```

shift = 'abcdefghijklmnopqrstuvwxyz'
randomChars = '1234567890qwertyuiopasdfghjklzxcvbnm,<.>;@[{]}#~/?'
privateKey =
'532554365436519532052075326432543543654365436075436543654365436
5436590436543645636432604325432523578325390136257326472214'
decryptedMessage = ''
message = encryptedMessage
#character insertion
count = 0
for character in message:
    for number in key:
        encryptedMessage.remove(number+count)
        count = count+1
#caeser cipher
for character in message:
    for number in key:
        find character index in shift
        character = shift[index-number]
output(decryptedMessage)

```

Variable list

Variable name	Data type	Function
loginUsername	String	Will be set to the user entered username when logging in and will be checked against the usernames in the pre-existing accounts file.
loginPassword	String	Will be set to the user entered password when logging in where it will then be hashed. This will then be checked against the correct place in the hashed passwords file and see if they are the same.
registerUsername	String	Will be set to the username input on the register page and then checked to make sure it is NOT already in the usernames file. If not, this will pass as a suitable username.
registerPassword1	String	Set to the first password on the register page. If the username is accepted, this password will be checked against password 2 to check if they're the same.
registerPassword2	String	When checked against password 1, if they are the same, this will be hashed and added to the password file.
hashedPassword	String	Once password is entered on the login page, it is hashed and checked against the account hashed passwords file. The passwords are hashed so no actual passwords are stored.
messageText	String	The text that the player would enter into notepad which would then be encrypted and decrypted
encryptBy	Integer	This will be used in the actual encryption algorithm when shifting certain characters using the key
publicKey	String	Will be used by the client computer when encrypting a message as all PGP public keys will be made global across all clients to access.
privateKey	String	Will only be used by the server when decrypting a message as PGP private keys won't be shared with any users.
PGPlen	Integer	Will also be used during encryption and decryption when using PGP keys to change a message. The value of this variable will decide how many times a message is parsed through to change or revert it.
chatLog	String	The text that is sent into the global online chat. This is firstly sent to the server from a client computer and then is sent out to all connected clients.

Test plan

<u>Test</u>	<u>Expected Result output</u>	<u>Type</u>
LOG IN - no username - no password	Username not registered	Null
LOG IN - no username - password	Username not registered	Null
LOG IN - username - no password	Incorrect password	Null
LOG IN - username - password	Successful Login	Normal
LOG IN - username - wrong password	Incorrect password	Normal
REGISTER - no username - no password 1 - no password 2	Username not available	Null
REGISTER - no username - only 1 password	Both passwords must be same	Normal
REGISTER - no username - password 1 - password 2	Username not available	Null
REGISTER - username - no password 1 - no password 2	Password cannot be empty	Null
REGISTER taken username	Username already taken	Normal
REGISTER - username - password 1 - password 2	Registration successful	Normal
NOTEPAD empty text	Cannot encrypt/decrypt empty message	Null
NOTEPAD text over character limit	Message too long	Normal

NOTEPAD • text on character limit	Converts message	Boundary
CHAT • empty text	Cannot send empty message	Null

Development

Setting up the server

Connection

To begin, I am going to need to first code the server on which clients will connect to. As stated before, I will be using a two modules to assist me.

The first module I will be using is 'Socket', which will help in creating sockets to communicate between computers, and therefore be able to create the main server which will receive and send things to the clients in order to make my program online.

The second module is 'Threading', which will allow me to create and run multiple threads at once, meaning I can have the server completing actions concurrently, meaning I can communicate with all connected clients at practically the same time.

```
import threading
import socket

#-----

ipAddress=socket.gethostbyname(socket.gethostname())
#getting current IP address so I can set it as host IP for clients to connect
to

host = ipAddress #local host
port = 21212 #between 0-65535

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM) #creating the
server socket
server.bind((host, port)) #adding the current IP and custom port
server.listen() #makes the socket ready for accepting connections

print(f'Server running on IP:{ipAddress}')
```

I started off by importing both modules, and made the host IP the current one my computer was connected to, which I was able to use socket to get. As shown when I ran the code, the server was successfully created and could run on the IP.

```
Server running on IP:172.31.96.252
█
```

Server functions

Next I needed to create the two key parts of the server, communicating with clients (sending and receiving messages), and also the handling of clients (when they're joining and leaving the server).

I will also need to add two lists to manage the clients, which is their client number and their usernames linked with them when they log in

```
clients = []  
usernames = []
```

I will likely not use this exact code in the final version of the program as it doesn't do exactly what is wanted for the program, but I will rather use it temporarily for testing reasons to allow the server and client to communicate during development.

Messages

```
def broadcast(message):  
    for client in clients:  
        client.send(message)  
  
def handle(client):  
    while True:  
        try:  
            message=client.recv(1024) #1024 bytes  
            broadcast(message) #using the broadcast function, this will send  
the message from a user to all connected users
```

While a client is connected to the server, they will constantly be checked for if a message has been sent in the global chat. This code will likely be changed later on in time when I add in the chat UI but for now this is the base code which will be used. The broadcast function on the other hand will take a message when sent by a user and then broadcast (send to all clients) in the chatroom.

Leaving / Joining

```
except:  
    index=client.index(client)  
    clients.remove(client)  
    client.close  
    nickname = nicknames[index]  
    broadcast(f'{nickname} left the chat!'.encode('ascii'))  
    nicknames.remove(nickname)  
    break  
  
def receive():  
    while True:  
        client, address = server.accept()  
        print(f'Connected with {str(address)}')  
        client.send('LOGIN'.encode('ascii'))  
        nickname = client.recv(1024).decode('ascii')  
        nicknames.append(nickname)
```

```

clients.append(client)

print(f'Nickname of client is {nickname}')
broadcast(f'{nickname} joined the chat'.encode('ascii'))
client.send('connected to the server'.encode('ascii'))

thread = threading.Thread(target=handle, args=(client,))
thread.start()

```

As shown in the code, the except carries on from the try in the messages part of code. This is because if the server is not checking for messages from the client, they must have left the server. The receive() function will check for new connections to the server and print them out on the server's shell and update the client list.

Setting up the client

Connection

Firstly, I will again need to set up the code for the client to initially connect to the server. For this, it will require the custom port number (I used 21212 on the code for the server) and also the IP address of which the server is running on. To start, I will need to import socket and threading once again and write a few lines of code which will establish the connection to the server.

```

import socket
import threading

#-----

client=socket.socket(socket.AF_INET, socket.SOCK_STREAM) #creating client
socket
client.connect(('172.25.240.1', 21212)) #connecting using the servers IP and
the custom port number

```

Client functions

Unlike before, the client will only really need two main functions when communicating which is to send messages and variables (such as keys) and also receive messages (on the global chat). As clients will only be managed on the server, it isn't necessary to have a client list like it was before.

```

def receive():
    while True:
        try:
            message=client.recv(1024).decode('ascii') #receiving message from
server, not client

```

```

        if message == 'LOGIN':
            client.send(nickname.encode('ascii'))
        else:
            print(message)
    except:
        print('An error occurred')
        client.close()
        break

```

Firstly, I set up the code for which will allow the client to receive messages. We can adapt the code later on so that it will also be able to communicate variables between the client and the server (like encrypted messages) as the encryption algorithm will be done on the server side so no keys are held locally.

```

def write():
    while True:
        message=f'{nickname}: {input("")}'
        client.send(message.encode('ascii'))

```

Next, I added the code which will send things to the server. The first part adds the username to the message so that the server knows which client it has come from, and also people in the global chat will be able to see the sender. Like before, this code will also likely be later adapted so that it can send variables (such as the unencrypted message) to the server so it can be encrypted by keys held server-side.

Account creation GUI

Login page

To begin, I will need to import the Tkinter module as that is what I will be using for the graphical side of my program. Firstly, I will need to make a function which creates a canvas on which the login page will be set up.

```

import tkinter as tk
from tkinter import *

loginCanvas = tk.Canvas(root, width = 450, height = 350)
loginCanvas.pack()

```

Next, I need to add the boxes and buttons which the user will interact with to log in. These include the username box, the password box and the login button. I will also need the title of the program at the top.

```

usernameLogin = tk.Entry(loginCanvas)
usernameLogin.place(x = 210, y = 70, width=120, height=20)
passwordLogin = tk.Entry(loginCanvas)
passwordLogin.place(x = 210, y = 110, width=120, height=20)

label = tk.Label(root, text='Crypter')
label.config(font=('Helvetica bold',28))
loginCanvas.create_window(225, 30, window=label)

```

```
label = tk.Label(root, text='Username:')
loginCanvas.create_window(177,80, window=label)
label = tk.Label(root, text='Password:')
loginCanvas.create_window(177,120, window=label)
```

Firstly, I added in the two entry boxes (username and password) and set up the variables in which they would be held. After this, I added in the program logo and changed the size and font of it by using Tkinters '.config' function. Finally, I added two text labels next to the password and username boxes to show what they were.

```
passwordLogin = tk.Entry(loginCanvas, show="*")
```

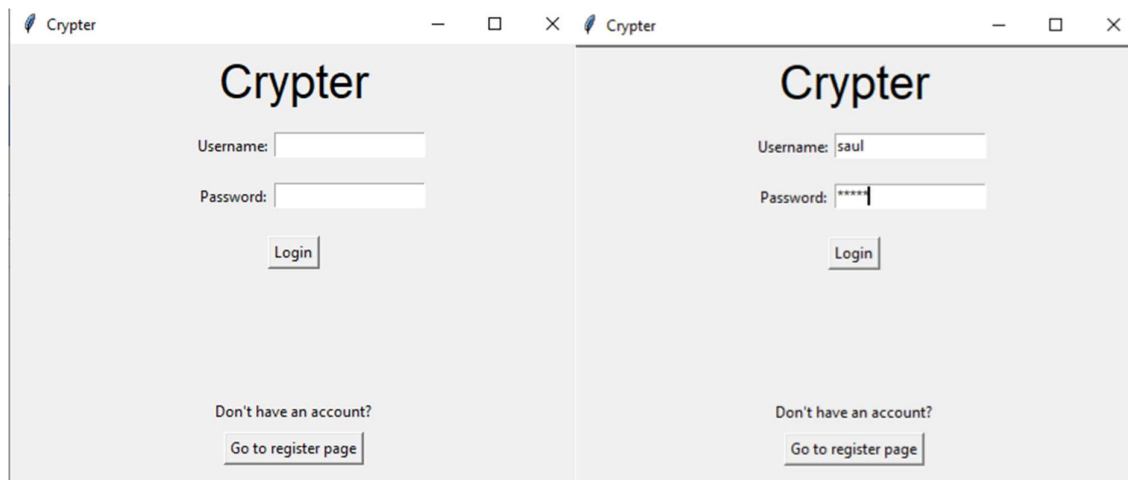
After the bare essentials were set up, I made a slight change to the password entry box code which would change the characters entered to '*'. This was a small change which would just make the overall appearance and professionalism of the program better.

Finally, I needed to add two buttons. One of these was the login button which would check the entered credentials, and the other would link over to the register page in case the client had not already signed in and created an account before.

```
loginButton = tk.Button(text='Login', command=loginCheck)
loginCanvas.create_window(225, 165, window=loginButton)

switchToLButton = tk.Button(text='Go to register page', command=registerPage)
loginCanvas.create_window(225, 320, window=switchToLButton)
```

Fairly simply, I added the two buttons which would run functions when pressed. These were the new 'loginCheck()' function and the 'registerPage()' function.



Register page

Once the registerPage function is ran (when the switch page button is clicked), I will have to first get rid of the login page and open the register page. The best way to do this that I thought of was to delete the login canvas and to create a new instance of the register canvas when needed.

```
def registerPage():
    loginCanvas.destroy()
    registerCanvas = tk.Canvas(root, width = 450, height = 350)
    registerCanvas.pack()
```

As before with the login page, I now need to add the buttons and boxes that will make it up. This will consist of a username box, two password boxes (one for confirming it was entered correctly) and a register box. I will also need to add a button that will link back to the login screen.

```
usernameRegister = tk.Entry(registerCanvas)
usernameRegister.place(x = 210, y = 70, width=120, height=20)
passwordRegister1 = tk.Entry(registerCanvas, show="*")
passwordRegister1.place(x = 210, y = 110, width=120, height=20)
passwordRegister2 = tk.Entry(registerCanvas, show="*")
passwordRegister2.place(x = 210, y = 150, width=120, height=20)
```

As before on the login page, I added in the entry boxes for the user. I will also need to create files for these to save information entered, but I will create these later when programming the login and register button functions.

```
label = tk.Label(root, text='Crypter')
label.config(font=('Helvetica bold',30), fg='#ABCDEF')
registerCanvas.create_window(225, 35, window=label)
label = tk.Label(root, text='Username:')
registerCanvas.create_window(177,80, window=label)
label = tk.Label(root, text='Password:')
registerCanvas.create_window(177,120, window=label)
label = tk.Label(root, text='Confirm Password:')
registerCanvas.create_window(155,160, window=label)
label = tk.Label(root, text='Already have an account?')
registerCanvas.create_window(225,290, window=label)
```

Once again to make everything clear to the user, I added labels around the page to point everything out. I also added the title before in the same place, font and size so it stays simple and consistent.

```
loginButton = tk.Button(text='Register', command=registerCheck)
registerCanvas.create_window(225, 205, window=loginButton)

switchToLButton = tk.Button(text='Go to login page', command=loginPage)
registerCanvas.create_window(225, 320, window=switchToLButton)
```

Finally, I added the register button for when all boxes are filled in and the user is ready to create an account which will create an account if the entry's pass all tests and it will then

redirect the user to login page automatically. This will run the register function when pressed. I also added the login page button if the user wants to swap back to log in.

The image displays two side-by-side screenshots of a web application window titled "Crypter". The window contains a registration form with the following elements:

- Username:** A text input field. In the left screenshot, it is empty. In the right screenshot, it contains the text "saul".
- Password:** A text input field with masked characters (*****). In the left screenshot, it is empty. In the right screenshot, it contains "*****".
- Confirm Password:** A text input field with masked characters (*****). In the left screenshot, it is empty. In the right screenshot, it contains "*****".
- Register:** A button located below the password fields.
- Already have an account?:** A text label located below the Register button.
- Go to login page:** A button located below the "Already have an account?" label.

Communicating user details

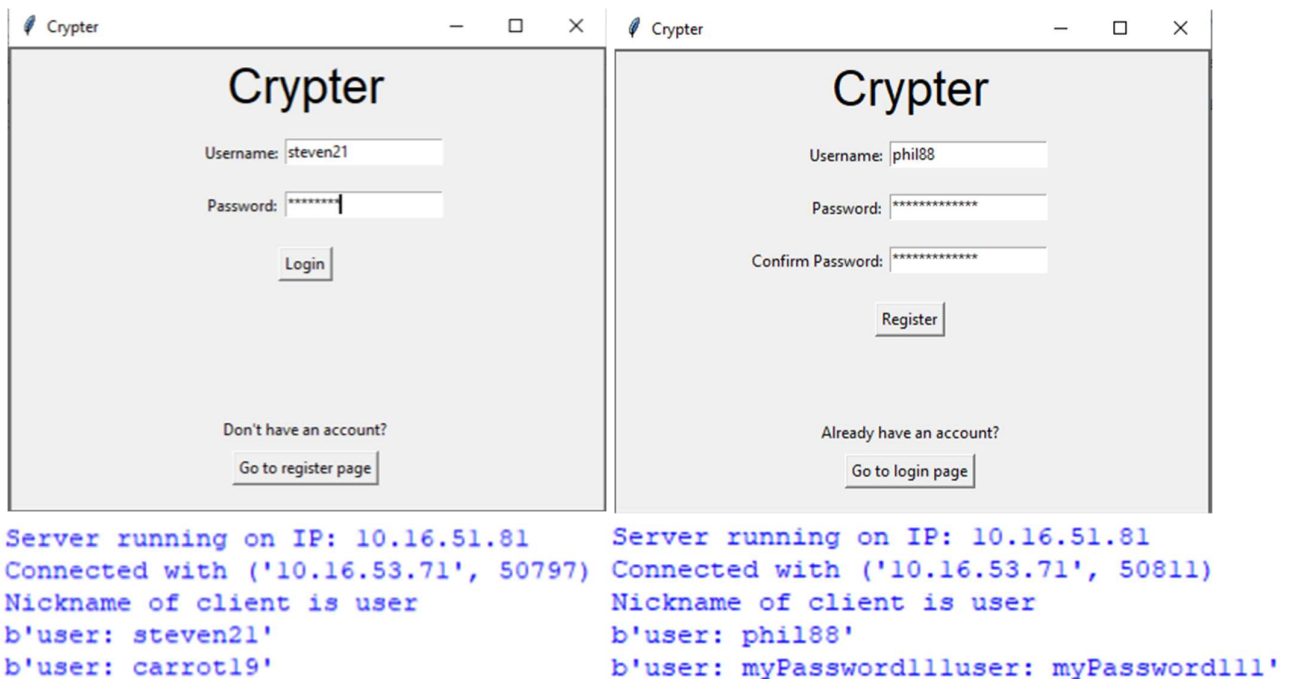
Method 1

```
def loginCheck():
    username=usernameLogin.get()
    password=passwordLogin.get()
    message=f'{nickname}: {username}'
    client.send(message.encode('ascii'))
    message=f'{nickname}: {password}'
    client.send(message.encode('ascii'))

def registerCheck():
    username=usernameRegister.get()
    password1=passwordRegister1.get()
    password2=passwordRegister2.get()
    message=f'{nickname}: {username}'
    client.send(message.encode('ascii'))
    message=f'{nickname}: {password1}'
    client.send(message.encode('ascii'))
    message=f'{nickname}: {password2}'
    client.send(message.encode('ascii'))
```

When the respective buttons are pressed (login and register), their functions are ran and the current data entered in the boxes will be messaged to the server.

Sending to server



The image displays two side-by-side screenshots of a web application titled "Crypter". The left window shows the login interface with fields for "Username" (containing "steven21") and "Password" (masked with "*****"), a "Login" button, and a link "Don't have an account? Go to register page". The right window shows the registration interface with fields for "Username" (containing "phil88"), "Password" (masked with "*****"), "Confirm Password" (masked with "*****"), a "Register" button, and a link "Already have an account? Go to login page".

Below the screenshots are two terminal windows showing the server's output. The left terminal shows the login attempt for user "steven21", and the right terminal shows the registration attempt for user "phil88".

```
Server running on IP: 10.16.51.81
Connected with ('10.16.53.71', 50797)
Nickname of client is user
b'user: steven21'
b'user: carrot19'
```

```
Server running on IP: 10.16.51.81
Connected with ('10.16.53.71', 50811)
Nickname of client is user
b'user: phil88'
b'user: myPasswordl1lluser: myPasswordl1l'
```

As shown on the server end when the register information is received, the password data is sent together despite the password messages being sent separate. As well as fixing this, I need to add something into the server code that will differentiate whether the data sent through is for a user registering or a user logging in.

I first attempted to add all of the necessary parts to a list which I called `dataList`, but quickly realised that socket (the module for the online aspect) does not allow list to be sent across machines, but rather only variables that are strings and integers. In order to fix this, I created a separate function which solely sends the data for registering or logging in and tried to add a loop that would individually send each item in the list.

```
def sendData(dataList):
    for item in dataList:
        client.send(item.encode('ascii'))

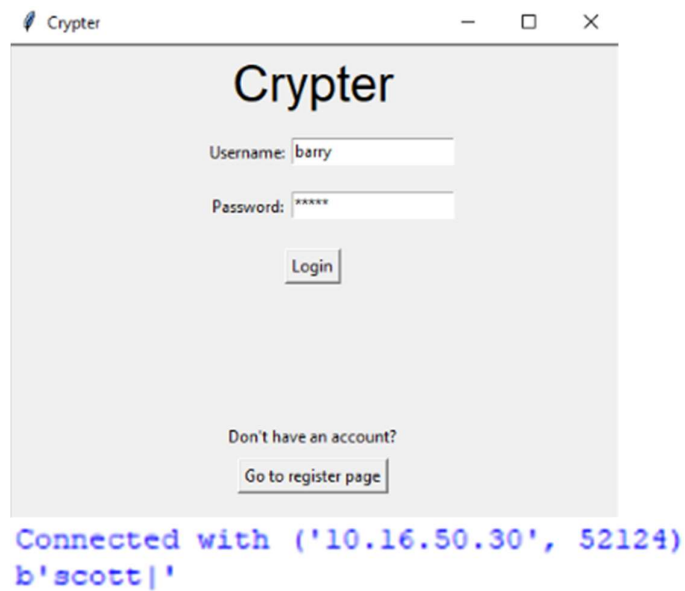
def registerCheck():
    client.send(usernameRegister.get().encode('ascii'))
    dataList = ['register', #identifier to show that list is for a registering
user
                usernameRegister.get(),
                passwordRegister1.get(),
                passwordRegister2.get()]
    sendData(dataList)
```

This however seemed to produce the exact same problem, and sent through the data as a block all in one line, rather than separate. The main necessity when sending over the user details was to send it as a block (which is why a list would have been ideal) as the server needs to be able to receive chunks to keep all data separate (e.g. keeping registering users details separate to logging in users details).

```
def sendData(dataList):
    dataString = ''
    for i in dataList:
        dataString=i+'|'
    client.send(dataString.encode('ascii'))

def loginCheck():
    dataList = ['login',usernameLogin.get(),passwordLogin.get()]
    sendData(dataList)
```

I changed the code so that all of the data would be sent as a string now, but each piece would be split apart by a specific characters so that it could be split up and stored as a list on the server side.



As shown, this was not printing all of the sent data out. I quickly located the problem and realised that rather than adding each piece to the data string, I was replacing everything in it. To fix this, I changed one line to add on to the string.

```
dataString+=i+'|'
```

```
Connected with ('10.16.50.30', 52140)
b'login|barry|scott|'
```

It was now printing out the intended result which meant this method would work also for the register system. Once this was all set up, the final code for the data sending looked like this:

```
def sendData(dataList):
    dataString = ''
    for i in dataList:
        dataString+=i+'|'
    client.send(dataString.encode('ascii'))

def loginCheck():
    dataList = ['login',
                usernameLogin.get(),
                passwordLogin.get()]
    sendData(dataList)

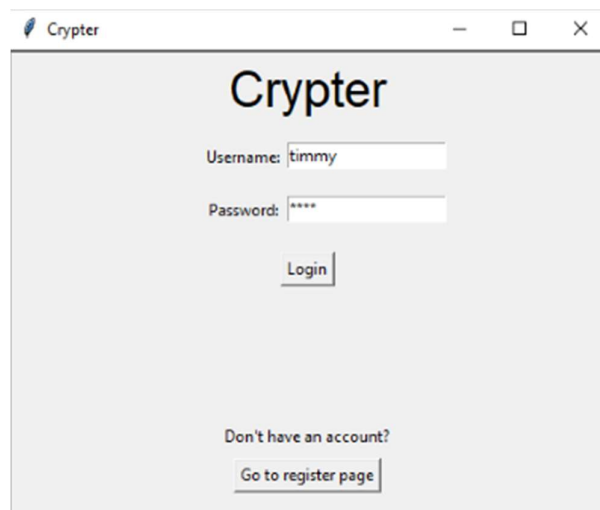
def registerCheck():
    dataList = ['register',
                usernameRegister.get(),
                passwordRegister1.get(),
                passwordRegister2.get()]
    sendData(dataList)
```

Receiving from client

Now this was all set, it meant moving on to the final stage which was splitting apart the data on the server side so that it could be stored in a list and eventually into separate files. This current issue is that all strings are being sent as byte strings, so this will have to be changed before the string is split up.

```
def broadcast(message):  
    print(message)  
    print(message[0])  
    message = str(message)  
    print(message)  
    print(message[0])  
    for client in clients:  
        client.send(message)
```

I used this code to test how the data would be received by the server. It would first print out the message while it is still stored as a byte string and then it will print out the first character. The code then turns the string into a normal ascii string and is again printed out.



```
Server running on IP: 10.16.49.183  
Connected with ('10.16.50.30', 52521)  
b'login|timmy|door| '  
108  
b'login|timmy|door| '  
b  
b'user left the chat!'  
117  
b'user left the chat!'  
b
```

As shown by the testing, the program could successfully turn the byte string into a normal string as shown by the first character changing from being 108 (the l in the string) into a b (the first character signifying the byte string).

```
def recieve(message):  
    message = str(message)  
    dataList = message.split('|')  
    print(dataList)  
  
recieve(message)
```

Now using the 'split' function in python, I will add to the server side code so that at each instance of the break character ('|'), it will split into a separate index of a list. I first tried this separately with a simpler string.

```
message = 'login|david|beckham|'
```

Output code:

```
['login', 'david', 'beckham', '']
```

This code now just needed to be adapted to work for byte strings that have been converted into a normal string.

```
message = "b'login|david|beckham|'"
```

Output code:

```
["b'login", 'david', 'beckham', "'"]
```

The fix for this is to limit the string two characters at the front (to cut off the b string signifier – b') and two characters at the end (to cut off the final apostrophe and the final break character - |').

```
message = "b'login|david|beckham|'"  
print(message[2:len(message)-2])  
message = message[2:len(message)-2]
```

Output code:

```
login|david|beckham  
['login', 'david', 'beckham']
```

Method 2

Though the first method would work, I didn't think it would be the best as using the break character as a splitting indicator meant that it couldn't be used in any data being sent between machines. Though I could limit its use in users' entries (not allowing the user to use it in their username, password or any message they want to encrypt), I wanted to refrain from doing this as I wanted the messages sent between users to be as free as possible.

This meant making another method of sending data which still allowed the use of lists. To start, I used the same code which would create the list containing the user's login or registration code, along with their username and password(s).

```
dataList = ['login',
            usernameLogin.get(),
            hashedPassword]
```

I know used a couple of separate built-in socket functions to help with the sending of lists. These were the 'sendall' method and the 'bytes' object. The 'sendall' method sends all of the data in a single call, unlike the 'send' method which may need to be called multiple times to send all of the data. The 'bytes' object is an immutable sequence of bytes which is similar to a 'str' object but it instead sends a sequence of bytes rather than characters.

```
server.sendall(bytes(str(dataList), "utf-8"))
```

When the string is decoded on the server side, it is received for example as:

```
b['login', 'john', 'myPassword25']
```

I therefore had to use the below code to revert the byte string back into the intended list so I can use that for the information checking.

```
message=client.recv(1024) #receives message server-side
dataString = message.decode("utf-8") #decodes message back into
byte string
dataString = dataString[1:-1].replace("'", "") #removes "[" and
removes all apostrophes
dataList = dataString.split(", ")#finally splits string at commas
into a list
```

The message is then converted back into the list:

```
['login', 'john', 'myPassword25']
```

This can now be manipulated so that the first part (list[0]) will tell the server what the client is sending over - whether its login data, registration data, a message to encrypt or decrypt or a message to put onto the chat.

Hashing

One thing that the client made clear was that hashing the passwords is important also. As mentioned before, the passwords will be hashed before being sent over to the server using a built in python module called 'hashlib'.

```
from hashlib import sha256
password='test'
hashed=sha256(password.encode('utf-8')).hexdigest()
print(hashed)
```

```
9f86d081884c7d659a2feaa0c55ad015a3bf4f1b2b0b822cd15d6c15b0f00a08
```

Now all I need to do is implement this module into the main client code so that the password will be hashed before being sent over the server. This will happen on both logging in and registering as when logging in, only the hashed password will be checked against the file, and when registering, the hashed password will be added to the file if account creation is successful.

```
from hashlib import sha256

def loginCheck():
    password = passwordLogin.get()
    password = sha256(password.encode('utf-8')).hexdigest()
    dataList = ['login',
                usernameRegister.get(),
                password]
    sendData(dataList)
print(dataList)
```

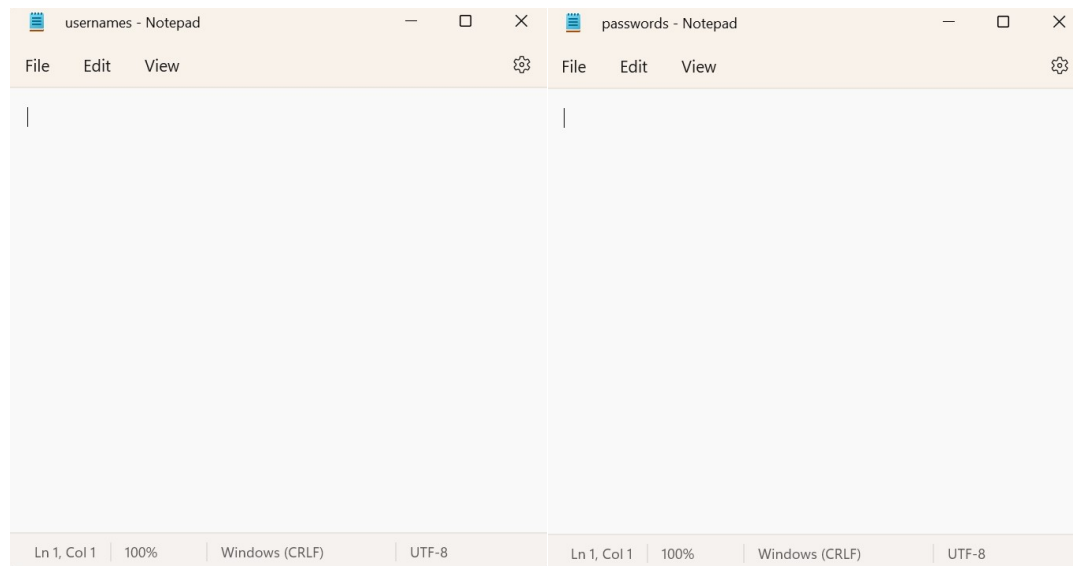
```
Connected with ('10.16.50.30', 54541)
b'login|david|ca978112ca1bbdcafacc231b39a23dc4da786eff8147c4e72b9807785afee48bb'
['login', 'david', 'ca978112ca1bbdcafacc231b39a23dc4da786eff8147c4e72b9807785afee48bb']
```

```
username = dataList[1]
password = dataList[2]
```

```
Connected with ('10.16.50.30', 54578)
b'login|david|ca978112ca1bbdcafacc231b39a23dc4da786eff8147c4e72b9807785afee48bb|ca978112ca1bbdcafacc231b39a23dc4da786eff8147c4e72b9807785afee48bb'
['login', 'david', 'ca978112ca1bbdcafacc231b39a23dc4da786eff8147c4e72b9807785afee48bb', 'ca978112ca1bbdcafacc231b39a23dc4da786eff8147c4e72b9807785afee48bb']
```

```
username = dataList[1]
password1 = dataList[2]
password2 = dataList[3]
```

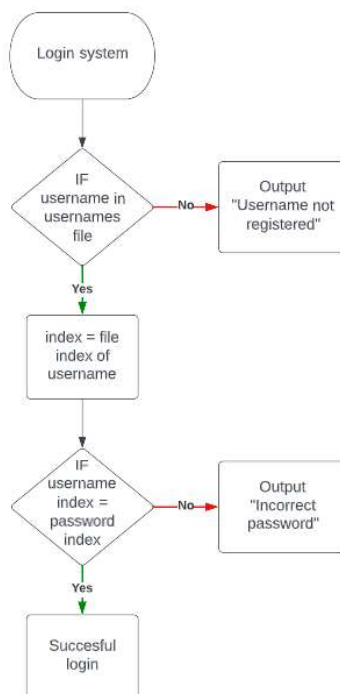

Setting up files



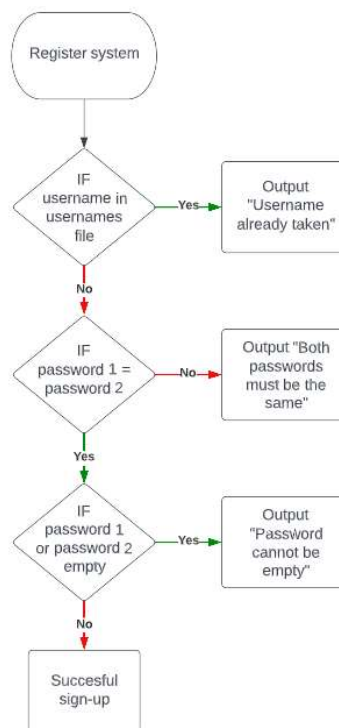
'usernames.txt'

'passwords.txt'

Log in system



Registering system



I will firstly need to create a function that check all of the files' contents and save them when a user either logs in or registers. These will be the usernames, passwords and eventually the key files.

```
def checkFiles():
    with open('usernames.txt') as file:
        usernames = file.read().split('\n')

    with open('passwords.txt') as file:
        passwords = file.read().split('\n')

    return usernames, passwords
```

I also added in a line that would remove any empty entries in the file when it is turned into a list. This would be beneficial so no indexes are out of sync between the files (username 4 will always be linked to password 4).

```
usernames = [x for x in usernames if x]
```

The first time that the server will need to interact with users is when a user has either attempted to log in or attempted to register an account. In the case of a client logging in, the information will need to be sent over to the server computer and checked against files containing usernames and passwords.

```
if username in usernames and username!='':
    place=usernames.index(username)
    if password==passwords[place]:
        #“ACCEPT USER”
    else:
        #“INCORRECT PASSWORD”
else:
    #“USERNAME NOT REGISTERED”
```

When registering however, the users inputted data will need to be checked as not already being in the username file, so that we know the name isn't already taken. The passwords will also need to be sent over so they can be hashed and stored in the passwords file if the account registers successfully.

```
if username not in usernames and username!='':
    if password1!=password2:
        #“BOTH PASSWORDS MUST BE SAME”
    elif password1==' ' or password2==' ':
        #“PASSWORDS CANNOT BE EMPTY”
    else:
        #“ADD DETAILS TO FILES”
        #“ACCOUNT CREATED”
else:
    #“USERNAME TAKEN”
```

Once the username and passwords have been sent over to the server, they will be checked and/or added to their respective files.

Setting up keys

As well as the usernames and passwords files, I will need to create two more files to store each accounts private and public keys. These will be randomly generated to work with one another when an account is registered and will not change upon creation.

These will also only be stored by the server where they are created so that there is no possibility of private keys being accessed by clients. In addition to this, all encryption and decryption using keys will be executed server-side, so that is where we will start to write the key code.

```
from random import randint as rdi
keyLen=10 #var
PGPlen=502+keyLen #var
```

I will first import the random module for the creation of keys. This will also be used when keys are being encrypted. I will also add to variables which I will for now set the values of as 502 and 10, so together when a key is assembled it will create the keys of length 512. This should be enough to encrypt any large size message a user will need to send.

```
def prodKEY():
#PUBLIC---
    publicKtoadd='-----BEGIN S.N PUBLIC KEY BLOCK-----\n\n'
    for i in range(PGPlen):
        publicKtoadd+=str(rdi(0,9))
    publicKtoadd+='\n-----END S.N PUBLIC KEY BLOCK-----\n'
```

For the producing of the public key, the program will loop the key length variable amount of times adding a random digit to the key each time. It will also but a key block at the start and end of the key.

```
#PRIVATE---
    privateKtoadd='-----BEGIN S.N PRIVATE KEY BLOCK-----\n\n'
    privateKtoadd+=publicKtoadd[KEYBLOCKLEN:KEYBLOCKLEN+keyLen]
    part=publicKtoadd[(KEYBLOCKLEN+keyLen):(PGPlen+KEYBLOCKLEN)]
    privateKtoadd+=part[len(part):-1]
    privateKtoadd+='\n-----END S.N PRIVATE KEY BLOCK-----\n'
    return publicKtoadd,privateKtoadd
```

for the private key, I will need to create a key which is different but works in a way that will reverse the public keys encryption. I will first create the key blocks to break up the key data. Now in order to create the key, I need to program an algorithm that will use the already generated public key.

The first thing to do is to create a constant that will determine the length of the start key block. In this case, the block “-----BEGIN S.N PRIVATE KEY BLOCK-----” plus the line break makes up a total of 38 characters.

```
KEYBLOCKLEN=38 #cons
```

The data of the public key is then taken from the key using the KEYBLOCKLEN constant as the border for the string. [KEYBLOCKLEN:KEYBLOCKLEN+keyLen] will split the string first at the 38th character, and then finish at the 38th+10th character splits the first 10 characters of the key off. This is because generally in public and private keys, there is a key which is similar between the two (e.g. at the start of PGP keys or the mod number in RSA encryption).

After this, the actual key part is taken from the public and is saved as a variable named 'part'. This is then finally reversed and added to the private key, along with the ending key block string. A final set of keys when produced looks like this:

-----BEGIN S.N PUBLIC KEY BLOCK-----

```
06458187944865083409731354268315193685902459195951964338161956877481128099
42742212787531617990308244641467668642392268978390952464828830407683315839
82816730207373732176510470154248754454244188211476342607983324644679742394
78275771491424296458787299694705906261446736103758271758578187025803616469
51019896854511105883181538346501262620919527346709644026772335657248737924
94492678338900047163925669159188343909309309440920171861511436882570832981
19888224599655137572569566174963511897985033254729758561099440148946066241
5136
```

-----END S.N PUBLIC KEY BLOCK-----

-----BEGIN S.N PRIVATE KEY BLOCK-----

```
06458187946315142660649841044990165857927452330589798115369471665965275731
55699542288891189238075288634115168171029044903903909343881951966529361740
00983387629449429737842756533277620446907643725919026262105643835181388501
11545869891015964616308520781875857172857301637644162609507496992787854692
42419417757287493247976446423389706243674112881442454457842451074015671237
37370203761828938513386704038828464259093879862293246866764146442803099716
13578721224724990821184778659161833469159591954209586391513862453137904380
5684
```

-----END S.N PRIVATE KEY BLOCK-----

As shown, the first 10 characters – '0645818794' – remain the same on both keys as this is the key identifier. After this however, the keys numbers change in a seemingly random way. Now both keys have been generated, they can be saved to their respective files on the server side.

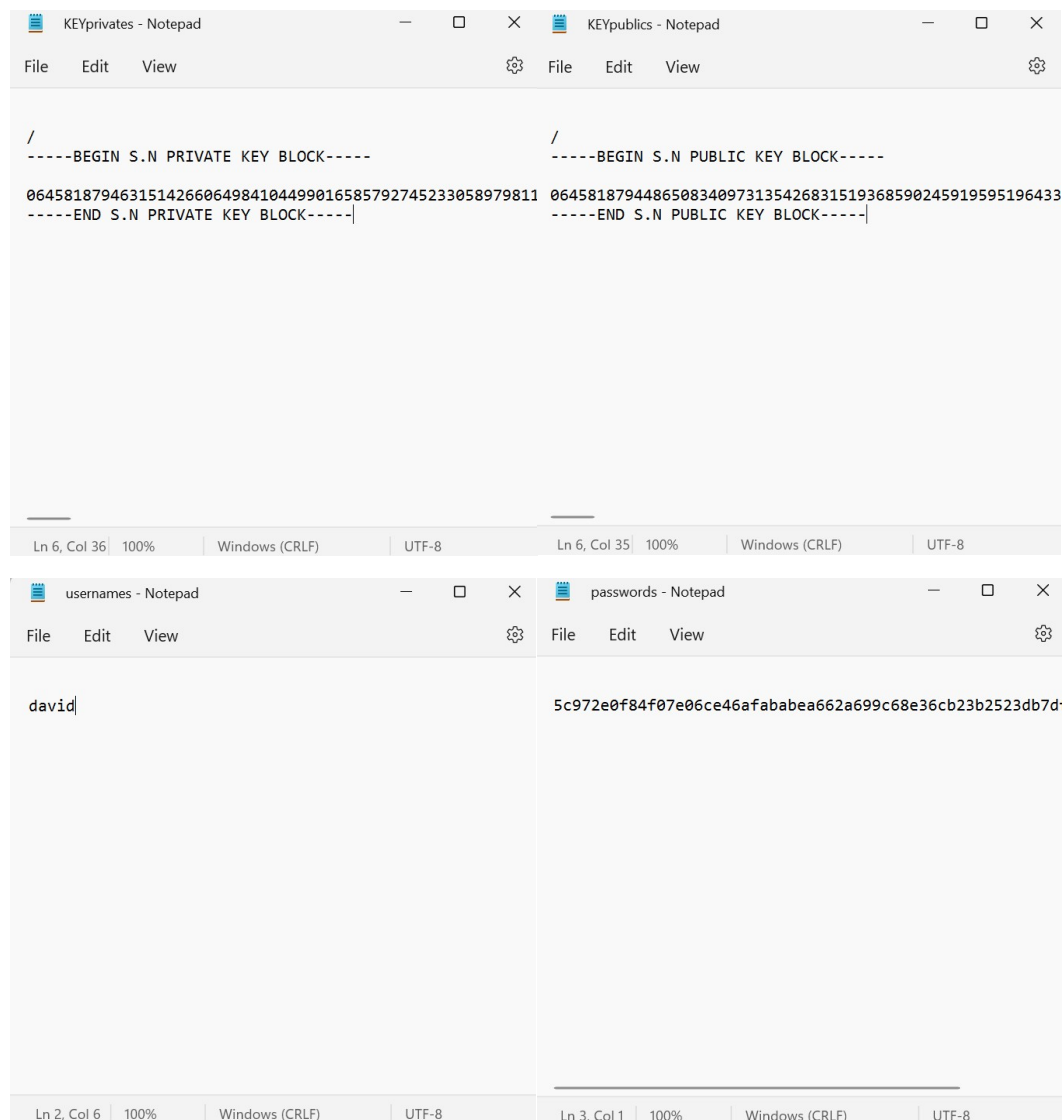
To do this, we simply have to run the prodKEY() function when an account is successfully created and save the returned public and private keys first as variables, and then to the KEYpublic.txt and KEYprivate.txt files.

```

publicKtoadd,privateKtoadd = prodKEY()
with open('usernames.txt','a') as file:
    file.write('\n'+username)
with open('passwords.txt','a') as file:
    file.write('\n'+password1)
with open('KEYpublics.txt','a') as file:
    file.write('\n/\n'+publicKtoadd)
with open('KEYprivates.txt','a') as file:
    file.write('\n/\n'+privateKtoadd)

```

Every time a username, password or key is added to a file it will need to be split up. For the usernames and passwords it is fine to do this just with a simple line break and they only take up a single line, but for the keys however, I will use a '/' character and the keys already line break themselves. This is why an extra character was added earlier in the KEYBLOCKLEN constant from 37 to 38.



Main GUI (server side)

Notepad

```
username = dataList[1]
password = dataList[2]

if username in usernames and username!='':
    place=usernames.index(username)
    if password==passwords[place]:
        message = ['ACCEPT',publics[place],privates[place],usernames]
        print(message)
        clients[0].sendall(bytes(str(message), "utf-8"))
```

Once a user has successfully logged in, they will be sent the above 'message' list in the same way that the log in and registration details were sent by the client. By using the variable place (the logged in users index in the files), their public and private key will be sent over, along with a list of all currently logged in users – 'usernames'.

The 'usernames' list will be used by the client in the drop down menu for when selecting who's public key they want to use to encrypt a message. Once this is chosen and a message has been entered, the user will use the decrypt button, in turn sending the code 'encrypt' and the message over to the server.

Encryption

```
elif dataList[0] == 'encrypt':
    encrypt(dataList)
```

When this is received, it will run the encrypt() function using the dataList sent over as a parameter, so that the message and public key they are using can be used by the server in the encryption algorithm.

```
fillList=['a', 'b', 'c', 'd', 'e', 'f',
          'g', 'h', 'i', 'j', 'k', 'l',
          'm', 'n', 'o', 'p', 'q', 'r',
          's', 't', 'u', 'v', 'w', 'x',
          'y', 'z', 'A', 'B', 'C', 'D',
          'E', 'F', 'G', 'H', 'I', 'J',
          'K', 'L', 'M', 'N', 'O', 'P',
          'Q', 'R', 'S', 'T', 'U', 'V',
          'W', 'X', 'Y', 'Z', '=', '+',
          ',', '.', '?', '@', '"', "'",
          '/', ' ', '1', '2', '3', '4',
          '5', '6', '7', '8', '9', '0']
shift="abcdefghijklmnopqrstuvwxyz"
```

I now created a list and a string which I will use during the encryption and decryption process. The first 'fillList' will be used to put in random characters into the message so that it appears

more random to the user and will be used dependant on the clients key. A similar method will also be used with the 'shift' string, which will move along the characters in a Caesar-shift like manner. This string could be changed to make it even harder to decrypt without a key, but for now I will leave it like this.

```
encryptBy=0
for char in message:
    key=int(privateKey[encryptBy])
    if char in shift:
        position = shift.find(char)
        new_position = (position + key) % 26
        new_character = shift[new_position]
        encryptedMessage += new_character
    else:
        encryptedMessage += char
    encryptBy+=1
```

For the character shifting, I will use a simple Caesar-shifting algorithm which shifts the character by whatever number is being parsed in the key. The for loop allows every character in the message to be parsed through, where it is then checked whether the shift will move it past the last character in the alphabet 'shift' string. If not, then it simply moves the character that many forward in the string to its new letter. If it is however, then it will use the modulus function to find how many characters over it is, and go from the start of the string.

```
ogTextLen=len(encryptedMessage)
count=0
for i in range(ogTextLen):
    for x in range(int(PublicKey [i])):
        ins=fillList[rdi(0,len(fillList)-1)]
        place=i+x+count
        encryptedMessage=encryptedMessage[:place]+ins+encryptedMessage[place:]
    count+=int(PublicKey[i])
    encryptedMessage+=' '
```

For the second encryption, the new message is again parsed through, but rather than shifting each character, new random characters from the fillList are used and added in front of the character. The amount of characters chosen is dependent on the number currently being parsed from the key. This is why two loops are used – one is to go through each character in the message, and the embedded one is to add the keys number amount of letters to the message.

```
encryptedMessage=('-----BEGIN S.N MESSAGE-----\n\n'+
    encryptedMessage+
    '\n-----END S.N MESSAGE-----\n')
```

Finally, I added the key block start and end onto the message so that once the message is put onto the chat, it can be successfully decrypted by another user using the same decryption algorithm.

Key:

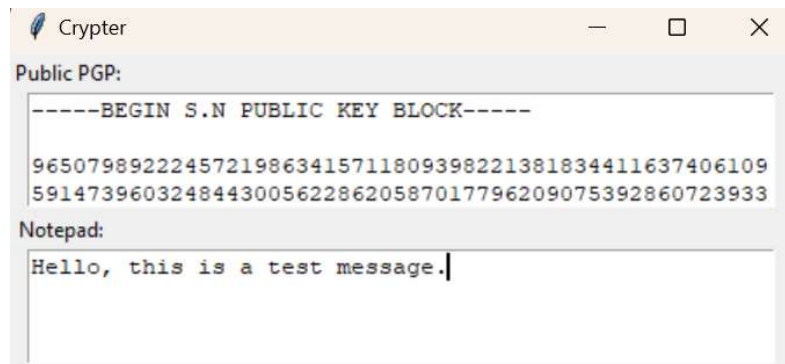
-----BEGIN S.N PUBLIC KEY BLOCK-----

```

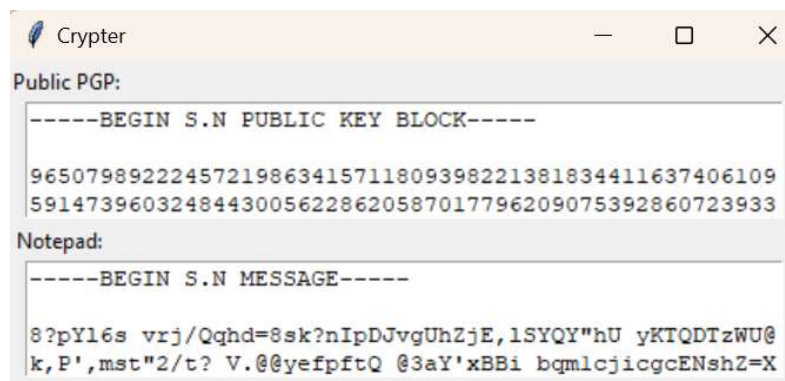
96507989222457219863415711809398221381834411637406109591473960324844300562286205870177962090753928
60723933377528409844781522945257902148018946001956483090409644025852606216812154202764432115574599
84708629890473373603395606584644134676187445952489207522916113722444791569160897335969817273331598
68403051255137173153008158018890089739818239989938573722652215449194378859748562414889518311063655
25105022241888117290873500238084529477500619350249111665960531564523840972899494796622331342958566
9976298113246267491222

```

-----END S.N PUBLIC KEY BLOCK-----

Message:

Hello, this is a test message.

Encrypted message:

-----BEGIN S.N MESSAGE-----

```

8?pYl6s vrj/Qqhd=8sk?nIpDJvgUhZjE,lSYQY"hU yKTQDTzWU@k,P',mst"2/t? V.@@yefpftQ
@3aY'xBBi bqmlcjicgcENshZ=X4LsbnAb/tx6JdMUbf4 QcolfnYGv7nfGi.="aUb'+Az= SoA61h4u+M.

```

-----END S.N MESSAGE-----

Decryption

For decryption, we will now use the reverse process, meaning we must first remove the fillList characters, and then do the reverse shifting. When a decryption message is sent through, the server also receives the cut down private key of the user who is trying to decrypt it.

```
numbersParsed=0
addCheck=0
while addCheck<len(message):
    addCheck+=int(privateKey[numbersParsed])+1
    numbersParsed+=1
for x in range(numbersParsed):
    for i in range(int(privateKey[x])):
        message = message[0 : x: ] + message [x+1 : :]
```

The variable addCheck will be added to every time characters are removed from the message, by the current number in the key. This means that when that number finally reaches the length of the message, all of the excess characters will have been removed and you will just be left with the shifted message.

```
decryptedMessage=''
decryptBy=0
for char in message:
    key=int(privateKey[decryptBy])
    if char in shift:
        position = shift.find(char)
        new_position = (position - key) % 26
        new_character = shift[new_position]
        decryptedMessage += new_character
    else:
        decryptedMessage += char
    decryptBy+=1
```

Reusing the same shifting code from before, we can just shift back in the string for each character the same way we shifted forward the amount of times for each number in the users private key. This overall will therefore be able to reverse the algorithm assuming the key used to encrypt was indeed the same users public key.

Chatroom

For the chatroom on the server side, it will need to be able to receive messages sent by the client. We will use the same method as for sending and receiving user login details to do this as this keeps the method consistent. As all the server needs to do is receive a message from one client and then send it to all connected, this will be a fairly simple task to code.

```
def chat(message):
    for client in clients:
        client.send(message)
```

This is all the code required to send the intended message out to all connected clients (who are held in a socket list called 'clients'). This message will then be received by the client and added to the chat text box, where users should be able to scroll through, and copy messages from to decrypt if intended for them.

Main GUI (client side)

Notepad

Now the files and keys have been set up, the main user interface is the next step for the project. As mentioned in the design section, this will need to include the notepad section and the chat section. The first section (notepad) is where users will enter messages and choose a public key to encrypt it with. The second section (chatroom) is where users will post and receive messages for them to decrypt and read, and after reply to.

```
def mainUI(place):
    global writingEntry, publicEntry

    mainCanvas = tk.Canvas(root, width = 450, height = 260)
    mainCanvas.pack()

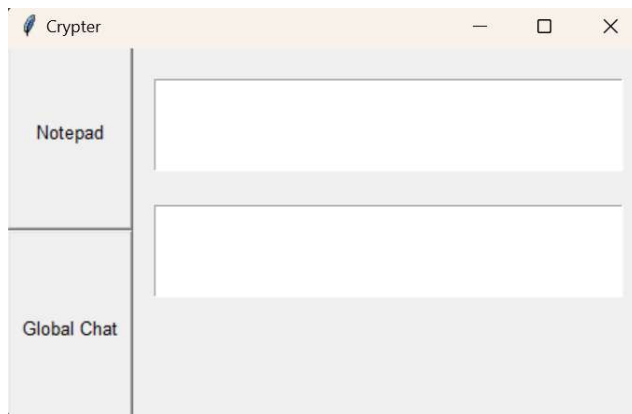
    writingEntry = tk.Text(root)
    writingEntry.place(x = 10, y = 112, width=430, height=66)
    publicEntry = tk.Text(root)
    publicEntry.place(x = 10, y = 22, width=430, height=66)
```

On the client side however, the user interface will open up. To create this, we first need to create the canvas for which the parts of the window will be added. Once this is done, we will add the two entry boxes, one where the user will input public keys, and one where the user will enter text to encrypt or decrypt.

```
notepadButton = tk.Button(text='Notepad', command=npCanvas,
width=10,height=8)
    notepadButton.config(font=('Helvetica bold',10), fg='black')
    mainCanvas.create_window(45, 60, window=notepadButton)

    chatButton = tk.Button(text='Global Chat', command=cCanvas, width=10,
height=8)
    chatButton.config(font=('Helvetica bold',10), fg='black')
    mainCanvas.create_window(45, 200, window=chatButton)
```

I also needed a system implemented which could switch between both the notepad and chat canvases. As Tkinter had no built in menus, I instead made two buttons and positioned them to the right side of the screen.



I now needed to add text to show what each entry box was and also the encrypt and decrypt buttons on the bottom. I will also change what the menu buttons look like so they differ from the encrypt and decrypt buttons. These will work in the same way to the switch page button on the login and register screen.

```
encryptButton.config(font=('Helvetica bold',10), fg='white', bg='grey')
```

This is simply done by changing the 'bg' (background) variable of it. I also made the 'fg' (foreground) variable white so the text stands out against the now darker grey background. Now this was changed, I could add in the code for the entry box labels and the encrypt and decrypt buttons.

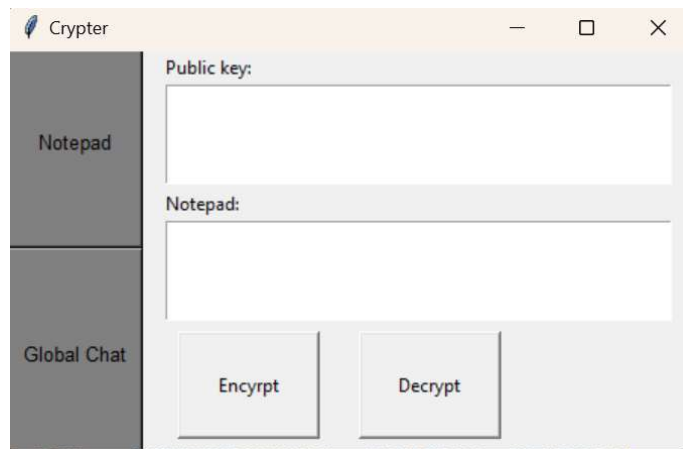
```
labelKey = tk.Label(root, text='Public key:')
mainCanvas.create_window(133, 10, window=labelKey)
labelNp = tk.Label(root, text='Notepad:')
mainCanvas.create_window(129, 100, window=labelNp)
```

Entry box labels placed above

```
encryptButton = tk.Button(text='Encrypt', command=encrypt, width=12, height=4)
mainCanvas.create_window(160, 220, window=encryptButton)

decryptButton = tk.Button(text='Decrypt', command=decrypt, width=12, height=4)
mainCanvas.create_window(280, 220, window=decryptButton)
```

Encrypt and decrypt buttons linked to functions 'encrypt()' and 'decrypt()'



The final thing to add the notepad part of the UI is the key menu. As mentioned before, this is where clients will select from a list of registered usernames. This will need to be sent over by the server as all data (like the currently updated usernames file) will be held server-side. This will be sent over every time a new user registers in the server-side notepad section so won't be a problem.

The first thing to do is the create a variable that will act as the front of the drop down menu. For this, we will leave it as the first username to have registered so that all indexes remain in order.

```
userVar = StringVar(root)
userVar.set(usernames[0])
```

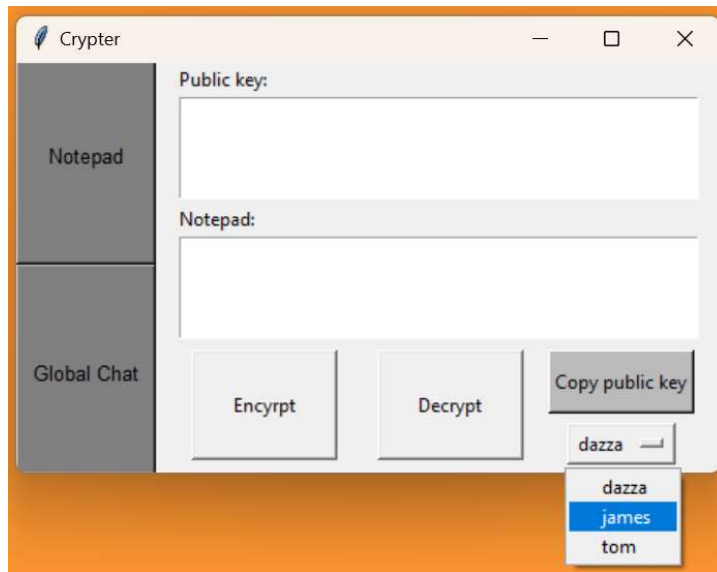
Once this is done, we can create then menu using that string variable as the front and the usernames list sent over from the server as the actual dropdown data.

```
menumask= OptionMenu(root, userVar, *usernames)
mainCanvas.create_window(390, 245, window=menumask)
```

Finally for the client notepad, we need to add the button that will copy the selected users public key. To do this, it will run a function - 'savePublicK()' - which automatically saves the key variable to the users clipboard through Tkinter.

```
copyButton = tk.Button(text='Copy public key', command=savePublicK,
width=12, height=2)
copyButton.config(bg='#BBBBBB')
mainCanvas.create_window(390, 205, window=copyButton)

def savePublicK():
    root.clipboard_clear()
    root.clipboard_append(publicKey[:-1])
```



Finished Notepad screen

Encrypt function

```
def encrypt():
    message=(writingEntry.get("1.0",tk.END+"-1c")).lower()
    publicKey=publicEntry.get("3.0",tk.END+"-37c")
    publicKey=publicKey[(keyLen+1):]

    dataList = ['encrypt',
                message,
                publicKey]
    server.sendall(bytes(str(dataList), "utf-8"))
```

As mentioned earlier, when the encrypt button is pressed the encrypt function is ran. As the main encryption will be done server side, the function of this button is very simple. First, it splits the data from the writing entry from the first line to the last character, which are parameters that need to be set in Tkinter, and then makes the writing into lowercase, which will make encrypting it later easier. This means that the message wont be changed.

For the publicEntry, it will start from the third line down (excluding the start key block) and end 37 charcaters before the last character in the key (excluding the end key block). The public keys key identifier is also split off using the 'keyLen' variable established earlier with key creation. This is how it will change a key:

Before:

-----BEGIN S.N PUBLIC KEY BLOCK-----

```
06458187944865083409731354268315193685902459195951964338161956877481128099427422127875316179903082
44641467668642392268978390952464828830407683315839828167302073737321765104701542487544542441882114
76342607983324644679742394782757714914242964587872996947059062614467361037582717585781870258036164
69510198968545111058831815383465012626209195273467096440267723356572487379249449267833890004716392
56691591883439093093094409201718615114368825708329811988822459965513757256956617496351189798503325
47297585610994401489460662415136
```

-----END S.N PUBLIC KEY BLOCK-----

After:

```
48650834097313542683151936859024591959519643381619568774811280994274221278753161799030824464146766
86423922689783909524648288304076833158398281673020737373217651047015424875445424418821147634260798
33246446797423947827577149142429645878729969470590626144673610375827175857818702580361646951019896
85451110588318153834650126262091952734670964402677233565724873792494492678338900047163925669159188
34390930930944092017186151143688257083298119888224599655137572569566174963511897985033254729758561
0994401489460662415136
```

Decrypt function

```
def decrypt():
    message=writingEntry.get("3.0",tk.END+"-28c")
    privateKey=privateKey[38:-38]
    privateKey=privateKey[(keyLen+1):]
    privateKey=privateKey[len(privateKey):-1]
    dataList = ['decrypt',
                message,
                privateKey]
    server.sendall(bytes(str(dataList), "utf-8"))
```

For decryption, the message they want to decrypt and the specific clients private key will be sent over to the server as the necessary parts for the decryption process. Before this happens, the message is split so that the start and end message blocks are split off. Also for the private key, the key blocks are split off, along with the first 10 characters which make up the key identifier which is made during key creation. Finally, the key is reversed so that it can be used in the decryption process to work in the opposite way to the public key. Here is an example of the process:

Before:

-----BEGIN S.N PRIVATE KEY BLOCK-----

```
06458187946315142660649841044990165857927452330589798115369471665965275731556995422888911892380752
88634115168171029044903903909343881951966529361740009833876294494297378427565332776204469076437259
19026262105643835181388501115458698910159646163085207818758571728573016376441626095074969927878546
92424194177572874932479764464233897062436741128814424544578424510740156712373737020376182893851338
67040388284642590938798622932468667641464428030997161357872122472499082118477865916183346915959195
42095863915138624531379043805684
```

-----END S.N PRIVATE KEY BLOCK-----

After:

```
63151426606498410449901658579274523305897981153694716659652757315569954228889118923807528863411516
81710290449039039093438819519665293617400098338762944942973784275653327762044690764372591902626210
56438351813885011154586989101596461630852078187585717285730163764416260950749699278785469242419417
75728749324797644642338970624367411288144245445784245107401567123737370203761828938513386704038828
46425909387986229324686676414644280309971613578721224724990821184778659161833469159591954209586391
5138624531379043805684
```

Reversed:

```
48650834097313542683151936859024591959519643381619568774811280994274221278753161799030824464146766
86423922689783909524648288304076833158398281673020737373217651047015424875445424418821147634260798
33246446797423947827577149142429645878729969470590626144673610375827175857818702580361646951019896
85451110588318153834650126262091952734670964402677233565724873792494492678338900047163925669159188
34390930930944092017186151143688257083298119888224599655137572569566174963511897985033254729758561
0994401489460662415136
```

We can now see the process has worked as both keys now are the same.

Chatroom

Before graphically creating the chat UI, I wanted to make a change to the switching page buttons where they would change colour dependant on whether you are on that page. To do this, I just changed what colour the buttons would be generated as when you are on the specific page.

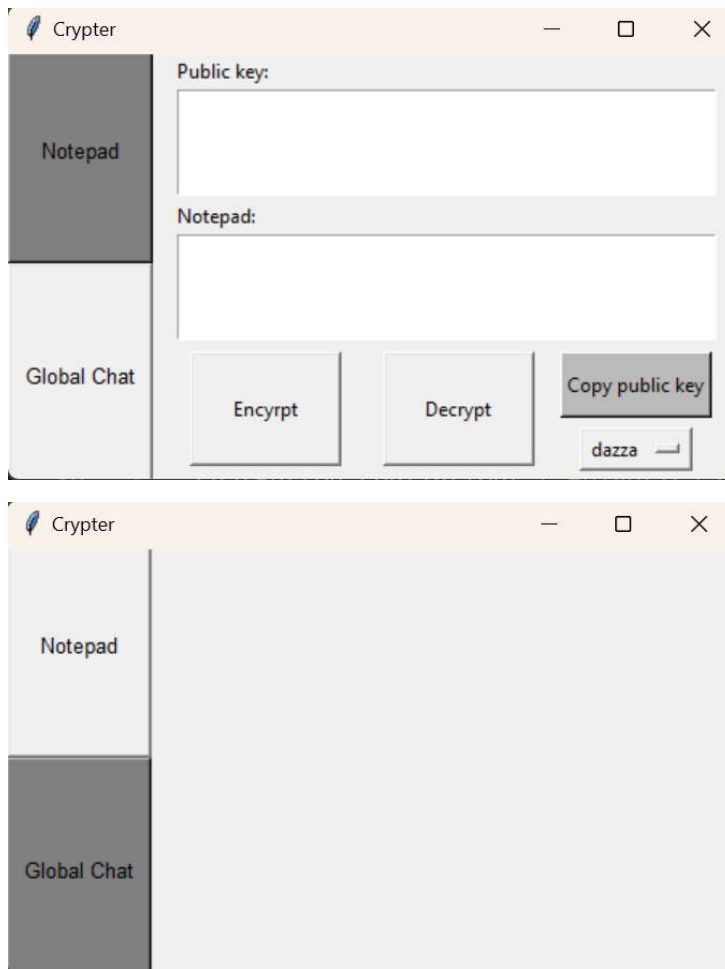
Chat page:

```
chatButton.config(font=('Helvetica bold',10), fg='black', bg='grey')
cCanvas.create_window(45, 200, window=chatButton)
notepadButton.config(font=('Helvetica bold',10), fg='black')
cCanvas.create_window(45, 60, window=notepadButton)
```

Notepad page:

```
chatButton.config(font=('Helvetica bold',10), fg='black')
npCanvas.create_window(45, 200, window=chatButton)
notepadButton.config(font=('Helvetica bold',10), fg='black', bg='grey')
npCanvas.create_window(45, 60, window=notepadButton)
```

This would make it so the the 'bg' colour would change to grey rather than the default light grey when the user was on that specific page, which added a simplistic but nice feel to the UI.



I now needed to add the widgets to the chat page as the final part of the GUI. This meant first adding the two buttons that would switch page, but I had already sorted that in the section before. Now the main window for the chat needed to be added, which I decided to use a text widget as if an entry widget was used to display text, then only one line of text can be displayed.

```
chatLog = tk.Text(cCanvas)
chatLog.place(x=95, y=10, width=330, height=210)
```

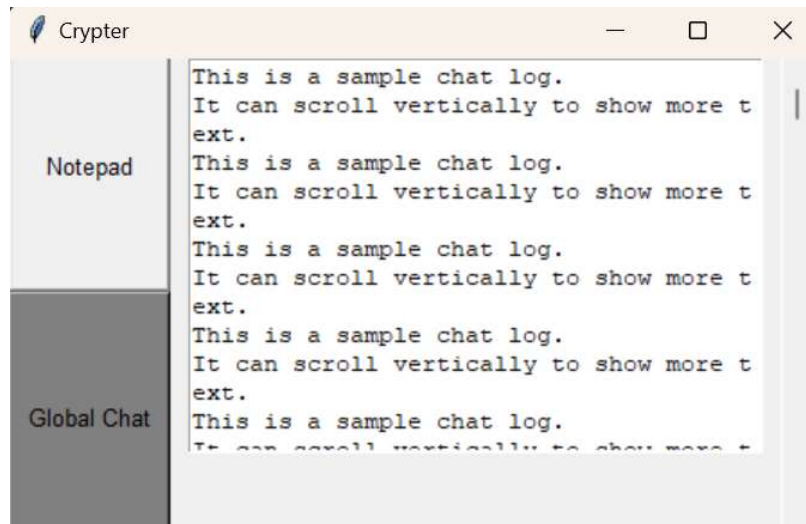
A scroll bar for the chat also needed to be added so that users can scroll back up to a message even if a new one has been sent. This will be necessary also as some of the messages will be large and therefore take up a lot of space when encrypted in the chat.

```
scrollbar = tk.Scrollbar(cCanvas)
scrollbar.place(x=430, y=0, height=220)

scrollbar.config(command=chatLog.yview)
```

Once the scrollbar was instantiated and positioned on the widget, I configured it so it was linked to the chatlog text widget and set it so it was a y scrolling bar so it would scroll the chat up and down. I then finally added some text to test the screen.


```
for i in range(10):
    chatLog.insert(tk.END, "This is a sample chat log.\n")
    chatLog.insert(tk.END, "It can scroll vertically to show more
text.\n")
```

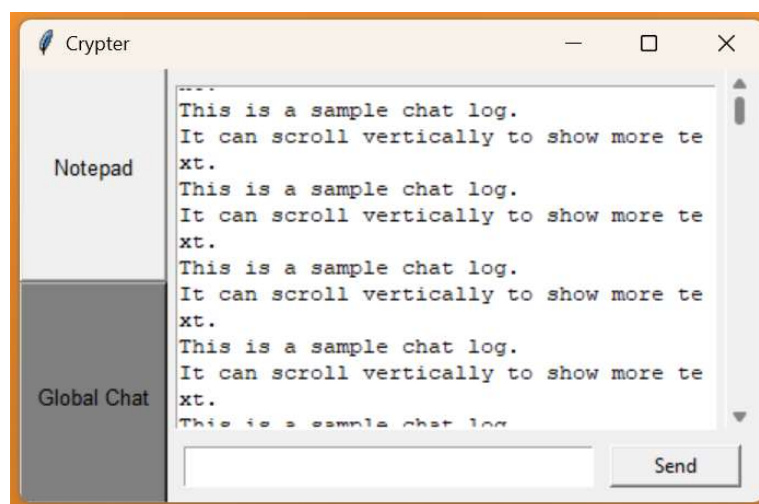


The page now only needed a message box and a send button which would link to a function, sending the entered message to the server. When pressed, the encrypted message will be sent server-side and there it will be sent out to all connected clients, and added to the chat on their screen when they open up the global chat.

```
messageEntry = tk.Text(cCanvas)
messageEntry.place(x = 100, y = 230, width=250, height=25)

sendButton = tk.Button(text='Send', command=sendMessage, width=10,
height=1)
cCanvas.create_window(400, 242, window=sendButton)
```

Button will run 'sendMessage' function.



Finished Chatroom screen

Final tweaks

Program logo

Something that I thought would make the program look more professional and independent was a logo to put on the GUI. In order to create these, I used AI to generate several encryption logos and then consulted with my client to see which he most preferred.

Logo 1:



Logo 2:



Logo 3:



After sending these icons to Joel, He said he would like to see how they look on the actual interface, and against the current colour scheme.

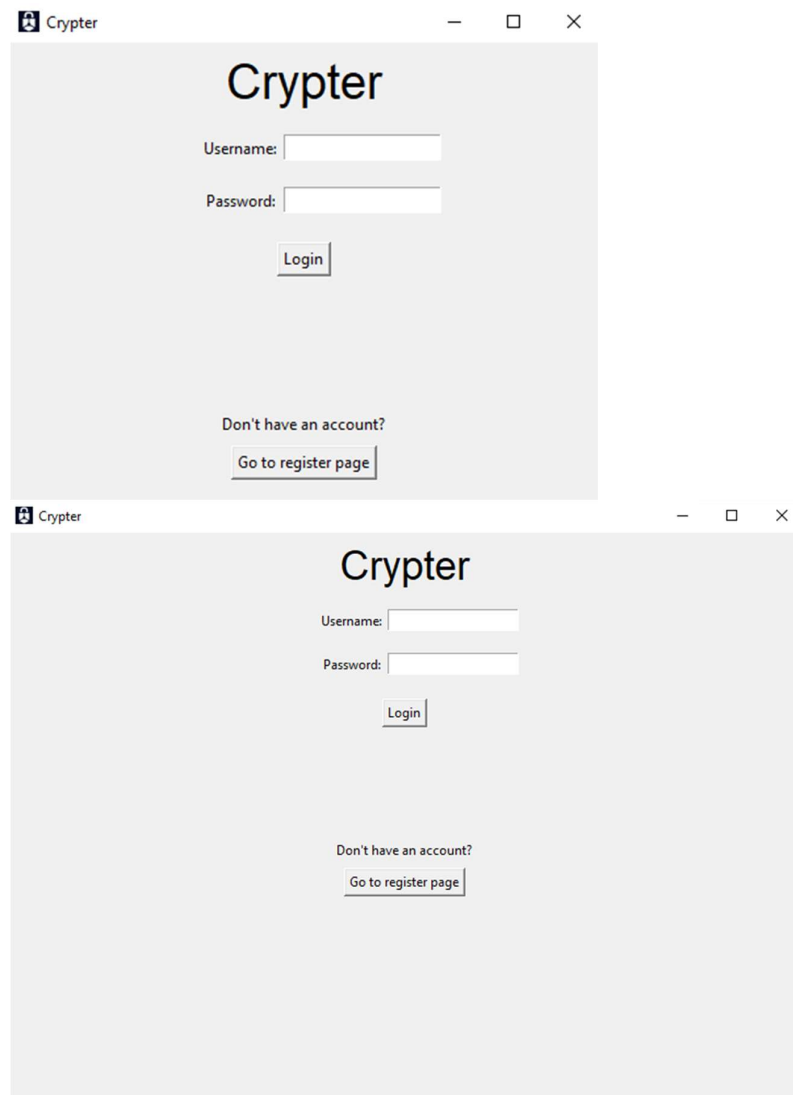


My client, Joel, ended up deciding that he liked the first logo the best, and said he was more than happy to include it in the program. This was simply done in Tkinter by using the 'iconbitmap' function, which allowed me to attach the logo as a .ico file to the canvas'.

```
root.iconbitmap('Crypter logo.ico')
```

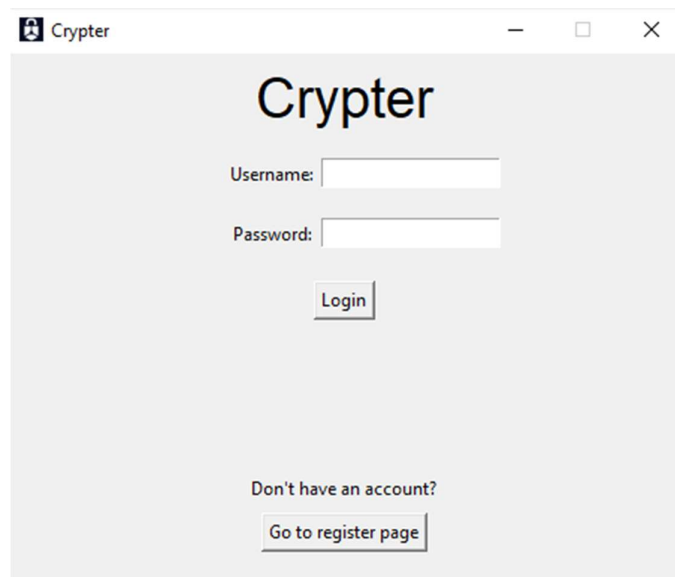
Resizing Tkinter window

Another very simple thing I wanted to change about the Tkinter root of the program was how resizable the canvas was. This was because all of the widget placements that I had set were for a specific window size which I have been using the whole time, and therefore the widget co-ordinates completely change with the window size.



As shown in the second, almost full-screen image, the aesthetics of the canvas completely change as the widgets stay in the middle and stay the same size, but the whitespace around increases. After a bit of searching around, I realised the quick fix would be very easy, and was just to change the availability of the full screen mode.

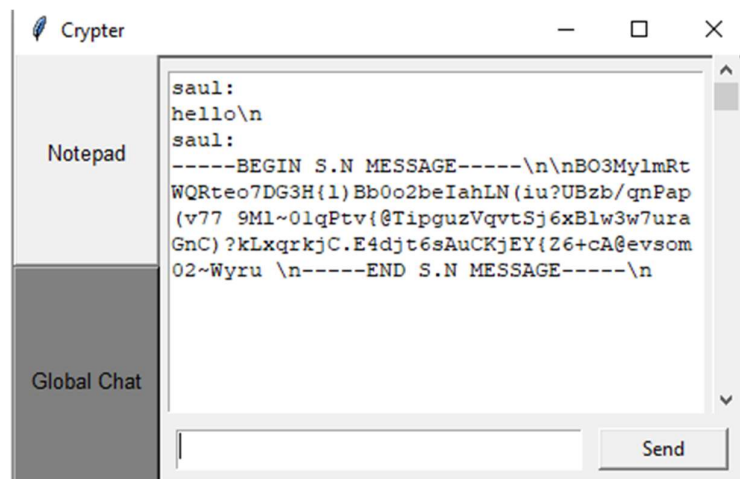
```
root.resizable(False, False)
```



User now unable to go full screen or resize the window in any way

Line breaks in global chat

One problem I noticed when having a look through the chatroom part of the program, was that the server side had issues interpreting the line breaks, and therefore was printing them out on the client side as their character value `'\n'`.

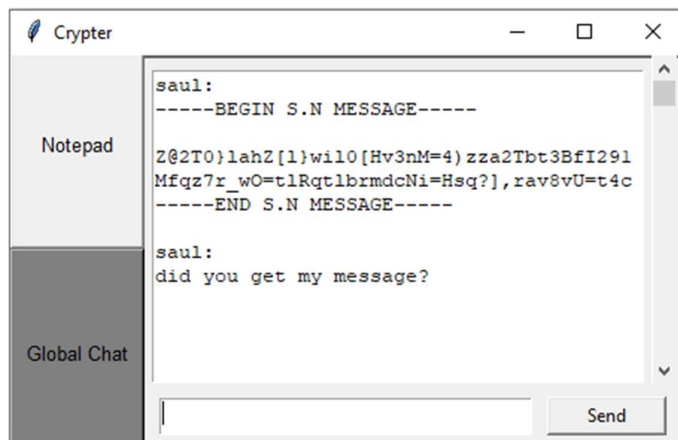


As shown, line breaks would still appear between the users username and their message, but these were the additional ones I added for whitespace. The original ones which should be the line breaks within the actual messages are still `'\n'` characters. In order to swap these for what they should be, I am using a similar method to what I used in the encryption of messages, which is splitting the string at `'\n'`s, and then replacing them with normal breaks between the lines. I then embedded this within the queue function so that it would allow me to insert back into the Tkinter chat widget.

```
def queueChat():
    while True:
        try:
            chat = qChat.get(block=False)

            chat = chat.split('\n')
            chatLines = ''
            for line in chat:
                chatLines += line+'\n'
            chatLines = chatLines[:-2]

            chatLog.delete('1.0', END)
            chatLog.insert(tk.INSERT, chatLines)
        except queue.Empty:
            break
    root.after(50, queueChat)
```



Line breaks now working

Redirecting to login page

One thing that my client Joel mentioned to me was that he would like a little feature where the user would be redirected back to the login page after successfully registering.

#Within the receiving loop

```
if dataList[0] == 'REGISTERED':
    print('Account registered! Redirecting...')
    registered = True
```

#After create account button is pressed

```
time.sleep(2) # Would be a wait in the redirecting
if registered == True:
    registerCanvas.destroy()
```

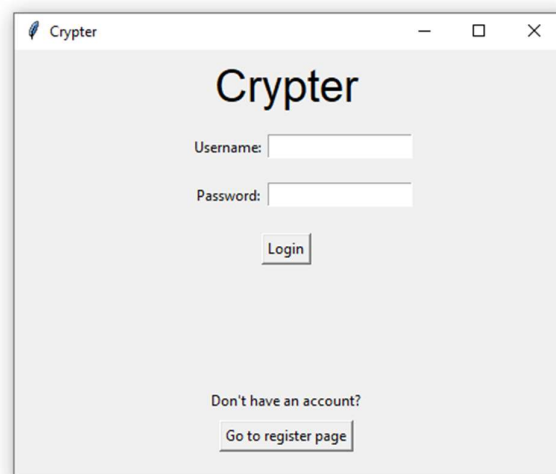
```
loginPage()
```

```
>>> Account registered! Redirecting...
```



A screenshot of a web browser window titled "Crypter". The page has a light gray background. At the top, the word "Crypter" is displayed in a large, bold, black font. Below it, there are three input fields: "Username:" with the text "Ed.Stephens", "Password:" with "*****", and "Confirm Password:" with "*****". Below these fields is a "Register" button. At the bottom of the form, there is a link that says "Already have an account?" followed by a "Go to login page" button.

```
>>> Account registered! Redirecting...
```



A screenshot of a web browser window titled "Crypter". The page has a light gray background. At the top, the word "Crypter" is displayed in a large, bold, black font. Below it, there are two input fields: "Username:" and "Password:". Below these fields is a "Login" button. At the bottom of the form, there is a link that says "Don't have an account?" followed by a "Go to register page" button.

User redirected to login screen

Printing messages on GUI

```

Username not available!
Password cannot be empty!
Passwords do not match!
Account registered! Redirecting...
Username not registered!
Incorrect password!
Login successful! Loading...
Invalid public key!

```

From a quick run through of the program, I found all of the statements which can be returned back to the user. From a quick check with Joel, my client, I was informed that he would rather this information to be displayed on the actual GUI, and the shell be left blank and unused on the client-side.

Output text	dataList[0]	Canvas to be printed on
Username not available!	""	Register canvas
Password cannot be empty!	""	Register canvas
Passwords do not match!	""	Register canvas
Account registered! Redirecting...	"REGISTERED"	Register canvas
Username not registered!	""	Login canvas
Incorrect password!	""	Login canvas
Login successful! Loading...	"LOGGEDIN"	Login canvas
Invalid public key!	""	Notepad canvas

Once identifying the functions where my output messages hwere, I needed to create another queue so that I could place text onto the respective canvas.

```

def queueOutput():
    while True:
        try:
            output = qOutput.get(block=False)
            print(output)

        except queue.Empty:
            break
    root.after(50, queueOutput)

```

```

if dataList[0] not in normalTerms:
    #single output messages to user
    qOutput.put(dataList[0])

```

Now I could use the user output outside of the receive function, I now had to program it to put the text on the correct canvas in the correct place. The three possible canvases are the login page (e.g., if a username is incorrect), the register page (e.g., if a username is taken) and the notepad page (if a public key is incorrect).

Similar to knowing when to place the chat log into the chat widget, I will use a try and except loop to attempt to place the text on a widget. If the text cannot be placed onto a widget, it therefore must not be the current canvas that is open for the client. Using try and excepts also greatly reduce the chance of errors and keep the program consistent.


```
coverLabel = tk.Label(root, text=' '*128) #whitespace label
outputLabel = tk.Label(root, text=output)

    try:
        loginCanvas.create_window(225, 190, window=coverLabel)
        loginCanvas.create_window(225, 190, window=outputLabel)
    except:
        try:
            registerCanvas.create_window(225, 235, window=coverLabel)
            registerCanvas.create_window(225, 235, window=outputLabel)
        except:
            try:
                npCanvas.create_window(395, 100, window=coverLabel)
                npCanvas.create_window(395, 100, window=outputLabel)
            except:
                pass #attempting to place output message on the three
possible widgets
```

Dependant on which widget the text inevitably gets placed on, the specific co-ordinates for the message are already set so the text is displayed in the correct position.

Testing

Checking test plan

The 4 sections in my test plan consisted of:

- The login page tests (attempted login checks)
- The registering page tests (attempted registering checks)
- The notepad page tests (cyphering message tests)
- The chat page test (blocking sending empty messages)

User login

<u>Test</u>	<u>Expected Result output</u>	<u>Type</u>
LOG IN • no username • no password	Username not registered	Null
LOG IN • no username • password	Username not registered	Null
LOG IN • username • no password	Incorrect password	Null
LOG IN • username • password	Successful Login	Normal
LOG IN • username • wrong password	Incorrect password	Normal

```
def loginCheck(dataList):
    usernames,passwords,publics,privates = checkFiles()

    username = dataList[1]
    password = dataList[2]

    print(dataList)

    if username in usernames and username!='':
        place=usernames.index(username)
        if password==passwords[place]:
            message = ['ACCEPT',publics[place],privates[place],usernames]
            print(message)
            clients[0].sendall(bytes(str(message), "utf-8"))
        else:
            message = ['Incorrect password']
            print(message)
```

```

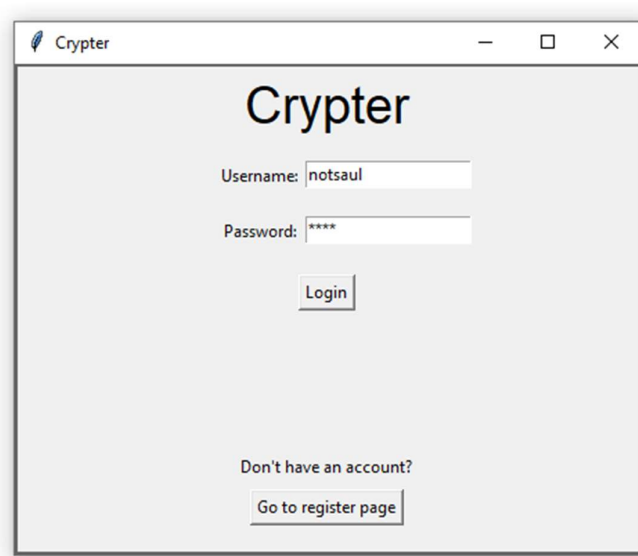
        clients[0].sendall(bytes(str(message), "utf-8"))
    else:
        message = ['Username not registered']
        print(message)
        clients[0].sendall(bytes(str(message), "utf-8"))

```

```

b"['Username not available']"
b"['Passwords do not match']"
b"['Account registered!']"
b"['Username not registered']"

```



```

['login', 'notsaoul', '45d25abffe8c792d74d30346429b5bc244b815eeb378a9c38395f7a466cf6894']
['Username not registered']

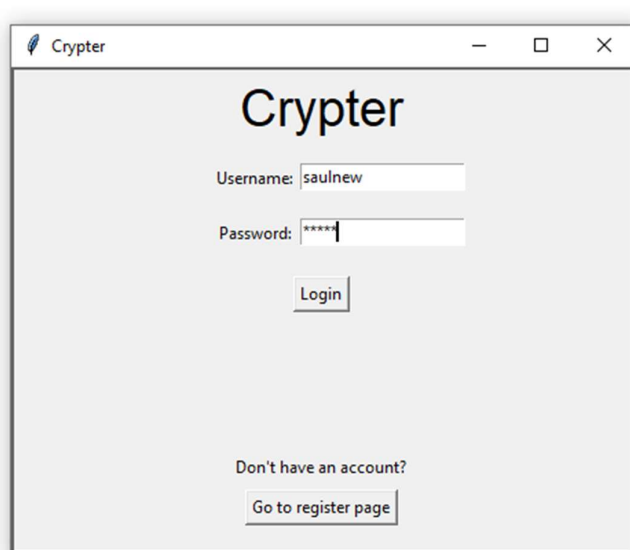
```

When a username is tried which is not in the file, the user will be prompted with a message telling then the tried username is not registered.

```

b"['Username not available']"
b"['Passwords do not match']"
b"['Account registered!']"
b"['Username not registered']"
b"['Incorrect password']"

```



```
['login', 'notsaul', '45d25abffe8c792d74d30346429b5bc244b815eeb378a9c38395f7a466cf6894']
['Username not registered']
['login', 'saulnew', '4915334ef7f882d7cbbd29d2006d27661525b64982500d402277fa7c396d4859']
['Incorrect password']
```

When a username that is registered however is tried but the password is not correct, a message will say so.

```
b"['Username not available']"
b"['Passwords do not match']"
b"['Account registered!']"
b"['Username not registered']"
b"['Incorrect password']"
b"['ACCEPT', 7, ['Guy_horne', 'saul', 'tm', 'sn', 'jmathews', 'sneedham16', '1', 'saulnew']]"
```



Finally, when the username and password are correct and their indexes match, the user will be accepted and logged into the Main UI, made up of the notepad and chat page.

User registration

Test	Expected Result output	Type
REGISTER - no username - no password 1 - no password 2	Username not available	Null
REGISTER - no username - only 1 password	Both passwords must be same	Normal
REGISTER - no username - password 1 - password 2	Username not available	Null
REGISTER	Password cannot be empty	Null

• username • no password 1 • no password 2		
REGISTER • taken username	Username already taken	Normal
REGISTER • username • password 1 • password 2	Registration successful	Normal

```
def registerCheck(dataList):
    print(dataList)
    usernames,passwords,publics,privates = checkFiles()

    username = dataList[1]
    password1 = dataList[2]
    password2 = dataList[3]

    if username not in usernames and username!='':
        if password1!=password2:
            message = ['Passwords do not match']
            print(message)
            clients[0].sendall(bytes(str(message), "utf-8"))
        elif password1==' ' or password2==' ':
            message = ['Password cannot be empty']
            print(message)
            clients[0].sendall(bytes(str(message), "utf-8"))
        else:
            publicKtoadd,privateKtoadd,keyID=prodKey()
            with open('usernames.txt','a') as file:
                file.write('\n'+username)
            with open('passwords.txt','a') as file:
                file.write('\n'+password1)
            with open('KEYpublics.txt','a') as file:
                file.write('\n\n'+publicKtoadd)
            with open('KEYprivates.txt','a') as file:
                file.write('\n\n'+privateKtoadd)

            message = ['Account registered!']
            print(message)
            clients[0].sendall(bytes(str(message), "utf-8"))
    else:
        message = ['Username not available']
        print(message)
        clients[0].sendall(bytes(str(message), "utf-8"))
```

```
b"['Username not available']"
```

```
Server running on IP: 10.16.49.83
Connected with ('10.16.49.58', 50571)
['register', 'saul', '7f82ec7e2862b56289ceaaeb057640d924ec668bbc3262444a441ale63
8ff139', '7f82ec7e2862b56289ceaaeb057640d924ec668bbc3262444a441ale638ff139']
['Username not available']
```

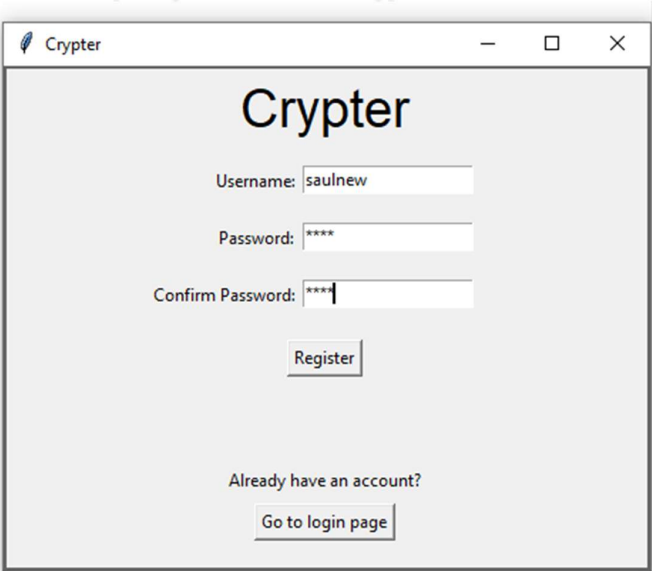
Trying the code with a username already in the list makes the server send a message to the client informing them that the username is already taken by an existing user.

```
b"['Username not available']"
b"['Passwords do not match']"
```

```
Server running on IP: 10.16.49.83
Connected with ('10.16.49.58', 50571)
['register', 'saul', '7f82ec7e2862b56289ceaaeb057640d924ec668bbc3262444a441ale63
8ff139', '7f82ec7e2862b56289ceaaeb057640d924ec668bbc3262444a441ale638ff139']
['Username not available']
['register', 'saulnew', '7f82ec7e2862b56289ceaaeb057640d924ec668bbc3262444a441a
e638ff139', '4e260b17a64aab76e936637a0f2a969a75a08e0e5d964e56cc6cbfb2782002dc']
['Passwords do not match']
```

Trying the code with registration details where both of the passwords do not match, the code again informs the client.

```
b"['Username not available']"  
b"['Passwords do not match']"  
b"['Account registered!']"
```



```
Server running on IP: 10.16.49.83  
Connected with ('10.16.49.58', 50571)  
['register', 'saul', '7f82ec7e2862b56289ceaaeb057640d924ec668bbc3262444a441ale638ff139', '7f82ec7e2862b56289ceaaeb057640d924ec668bbc3262444a441ale638ff139']  
['Username not available']  
['register', 'saulnew', '7f82ec7e2862b56289ceaaeb057640d924ec668bbc3262444a441ale638ff139', '4e260b17a64aab76e936637a0f2a969a75a08e0e5d964e56cc6cbfb2782002dc']  
['Passwords do not match']  
['register', 'saulnew', '45d25abffe8c792d74d30346429b5bc244b815eeb378a9c38395f7a466cf6894', '45d25abffe8c792d74d30346429b5bc244b815eeb378a9c38395f7a466cf6894']  
['Account registered!']
```

Finally, if the username is not already taken, and both hashed passwords match up with one another, the account will be registered and the data will be added to its files. Only one of the passwords (in the code it is password1) will need to be added to the files as they are the same and only a record of one is needed therefore.

At this point, there keys will be generated and added to the 'publicKey' and 'privateKey' files, at the same index as their name and password.

Cyphering messages

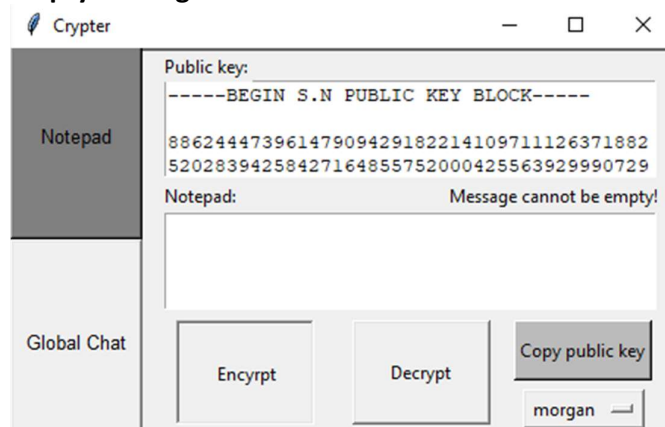
Test	Expected Result output	Type
NOTEPAD empty text	Cannot encrypt/decrypt empty message	Null
NOTEPAD text over character limit	Message too long	Normal
NOTEPAD text on character limit	Converts message	Boundary

Now according to my test plan, for the notepad section of the program I needed to check whether certain limitations would be in place when a message is encrypted/decrypted. The message the user is attempting to encrypt or decrypt is first sent over to the server, where it is checked, and a response message is sent back - just like on the login and register page.

To check for if the message follows a certain rule that the client has specified, the server side code will run if statements on it and then either allow the message to be encrypted and sent back, or will return a specific error message.

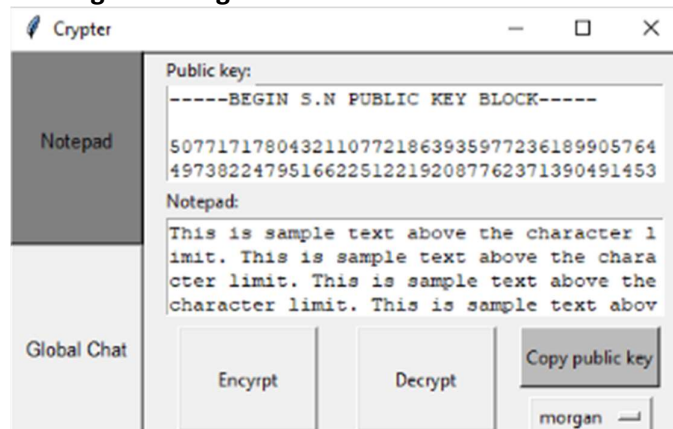
```
if len(text)!=0:
    if len(text)<=128:
        #encrypt message
    else:
        message = ['Message too long!']
        print(message)
        clients[index].sendall(bytes(str(message), "utf-8"))
else:
    message = ['Message cannot be empty!']
    print(message)
    clients[index].sendall(bytes(str(message), "utf-8"))
```

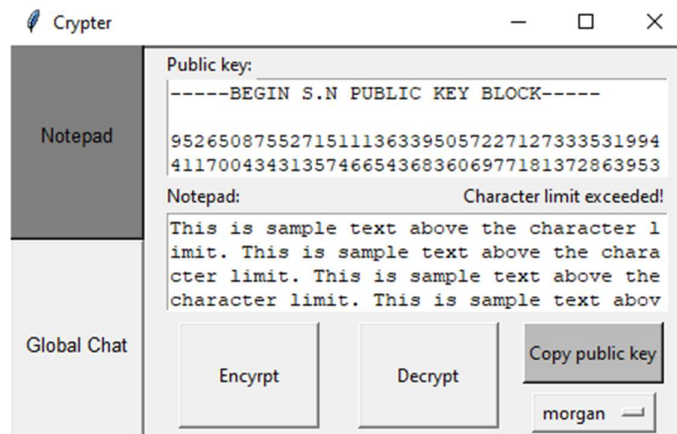
Empty message



When message is empty and attempted to be encrypted, an output is returned

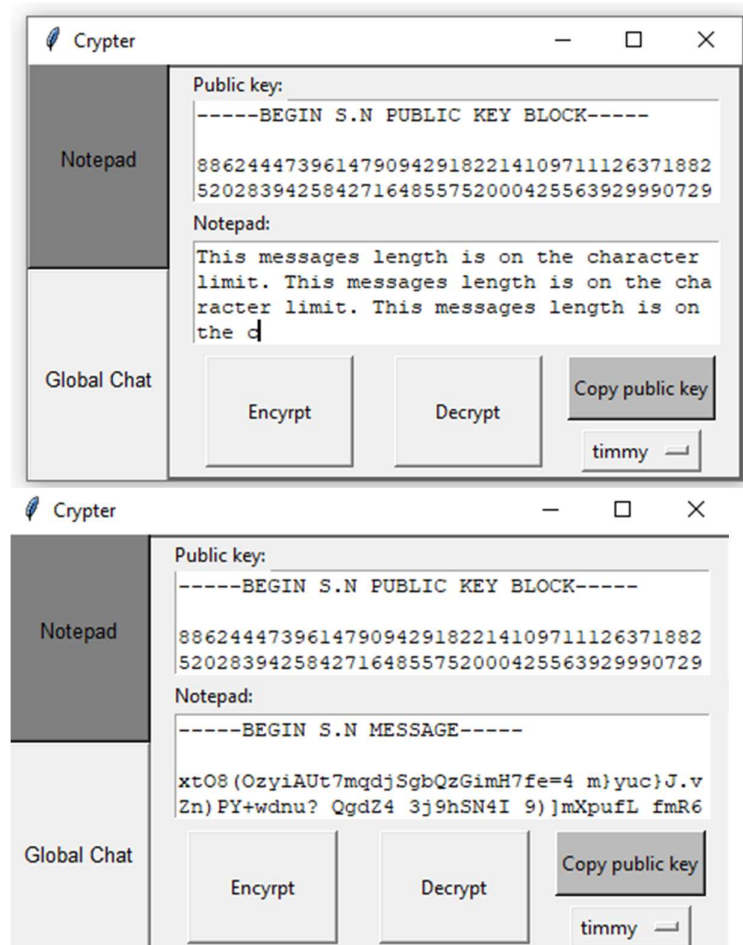
Message too long





When message is over the specified character limit and attempted to be encrypted, an output is returned

Message on character limit (boundary)

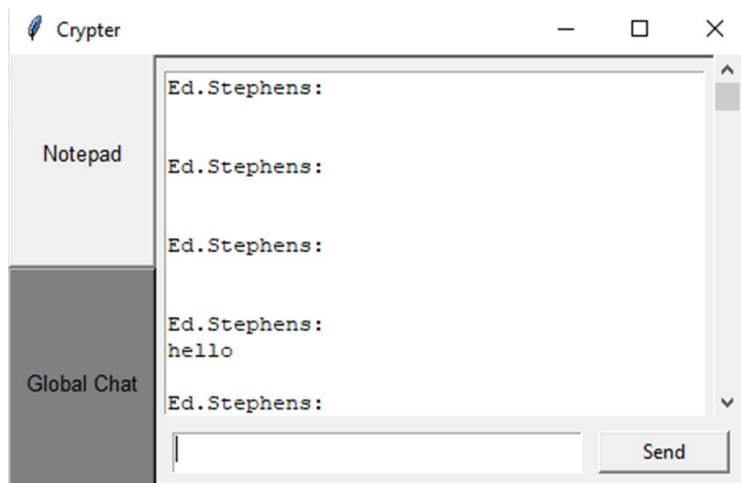


When message length is the same as the character limit, it is encrypted as expected. This means no output error message is sent to the user

Global chat

Test	Expected Result output	Type
CHAT empty text	Cannot send empty message	Null

One thing I hadn't thought about with the global chat was that the users would be able to send as many messages as they want, which would make the chat very susceptible to spam, which was made even easier by the fact that the user could enter empty messages. Making this small change by not allowing the user to do this would improve this feature overall I believed.



```
message = messageEntry.get("1.0",tk.END)
if message!='':
    #send message
```

After trying this small change which should have limited the ability to send empty messages, the problem was still occurring. To test why, I simply printed out what the message variable was at the time of sending. This showed that each message that was blank was actually being inputted as a line break, so I therefore just had to check for:

```
if message!='\n':
```

This now worked for empty messages, but not for ones that contained just whitespace, which looked the exact same when spam-sent into the global chat. I therefore did one final change to the checking if statement and used the 'isspace()' function to make sure it was false.

Final function code:

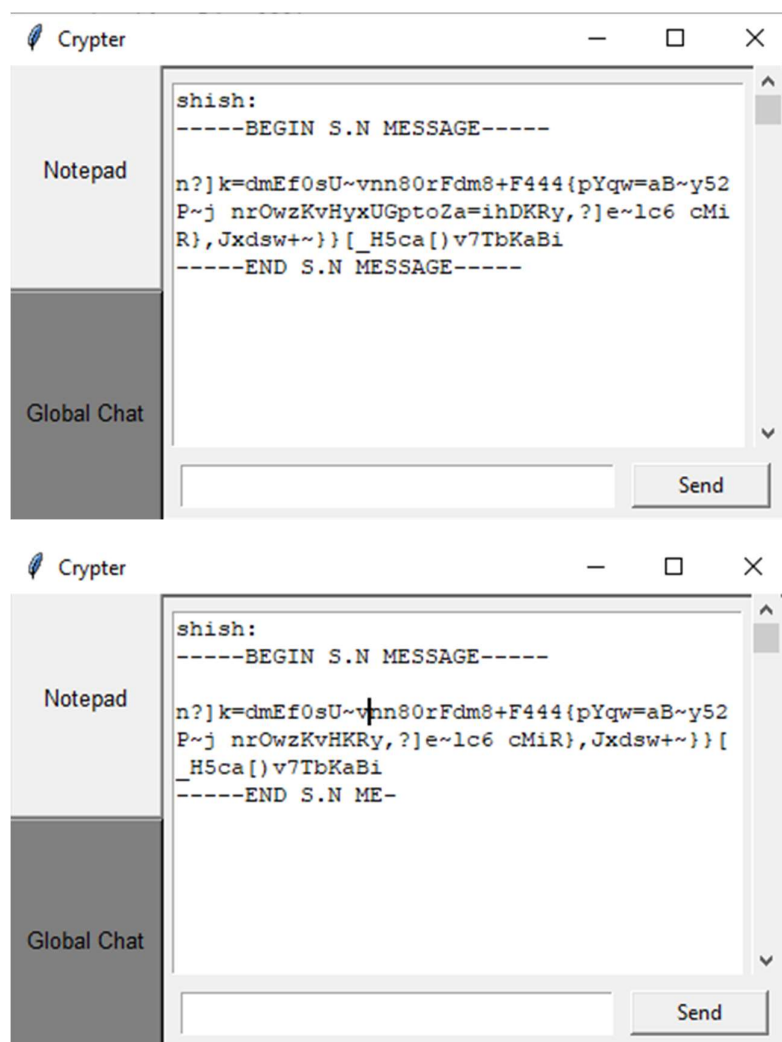
```
def sendMessage():
    message = messageEntry.get("1.0",tk.END)
    if message.isspace() == False:
        messageEntry.delete('1.0', END)
```

```
dataList = ['send',
            clientUsername,
            message]
server.sendall(bytes(str(dataList), "utf-8"))
```

Extra testing

Interaction with global chat

The first thing that I noticed when doing my final checks over the program was that the widget where the chat log is displayed was editable to the user. This would reset when the client would switch page and then go back onto the global chat however this was an issue as it meant the user could appear to change the chat on their side.



```
chatLog.delete('1.0', END)
chatLog.insert(tk.INSERT, chatLines)
chatLog.configure(state='disabled')
```

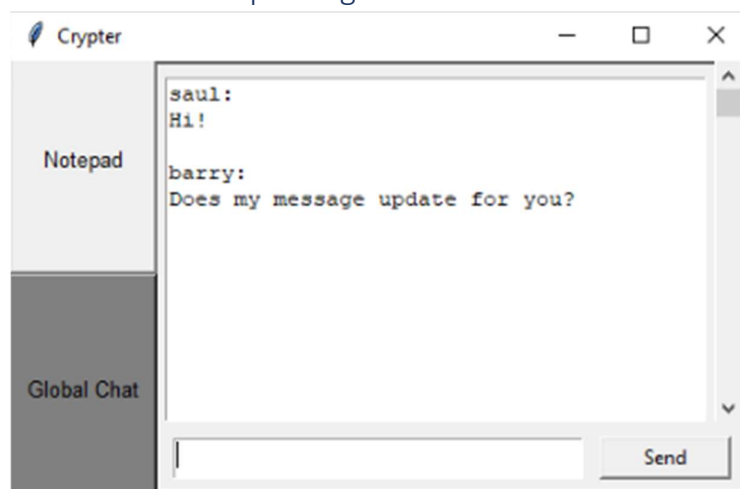
In order to prevent this, I tried adding a line of code at the end of writing in the chat log that would set its state to disabled. When I tested this, it did make the widget disabled and didn't

allow anything to be written into it, however it was not inserting the text into it so it was just a blank widget.

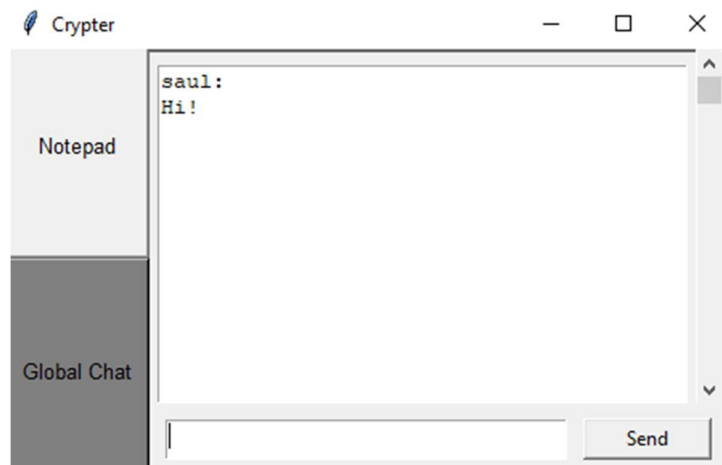
What was happening was the first time you go onto the global chat just to view it, it would set it to disabled and therefore, any other users' messages would not be able to be inserted as the widget would now be unchangeable. To get around this, I had to change the code so that the widget was enabled each time just before the new text is entered for the users.

```
chatLog.configure(state='normal')
chatLog.delete('1.0', END)
chatLog.insert(tk.INSERT, chatLines)
chatLog.configure(state='disabled')
```

Global chat updating



'barry's screen – 14:27

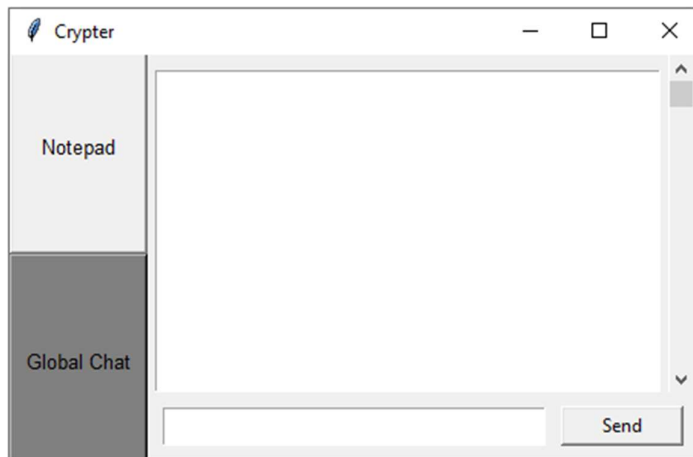


'saul's screen – 14:28

During testing the global chat with multiple users connected at once, I found that when a user sent a message, the chat would update for them, but not for all other users connected until

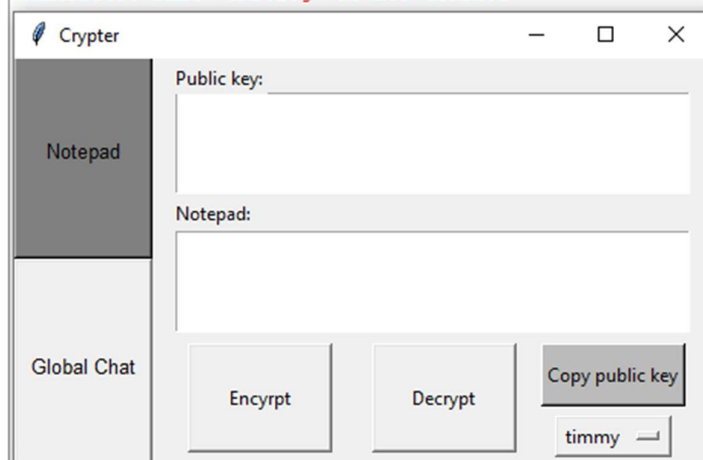
they had refreshed and gone back onto the global chat. This was not a major issue, but I still wanted the program to look and run as nicely as possible.

In order to fix this, I believed I would need to do a similar thing as I did with the clients username lists, which was updating the chat for every connected user, whether they were currently on the global chat screen or not.



'barry's screen – 14:36 (switch to global chat)

```
Exception in Tkinter callback
Traceback (most recent call last):
  File "C:\Program Files\Python310\lib\tkinter\__init__.py", line 1921, in __call__
    return self.func(*args)
  File "C:\Program Files\Python310\lib\tkinter\__init__.py", line 839, in callit
    func(*args)
  File "N:\My Documents\Year 13\Computing\Server\0 client.py", line 118, in queueChat
    chatLog.configure(state='normal')
NameError: name 'chatLog' is not defined
```



'saul's screen – 14:36 (already logged in, error)

As shown by the error, the client computer which currently had the notepad screen open could not determine what the chatLog widget was as that is on the other global chat canvas, and therefore it could not insert the chat anywhere. This meant that whenever a message is sent from a client or they go onto the chat screen, every connected user will be sent the full

chat log and the client will attempt to display it, which is erroneous if they are not on the chat screen.

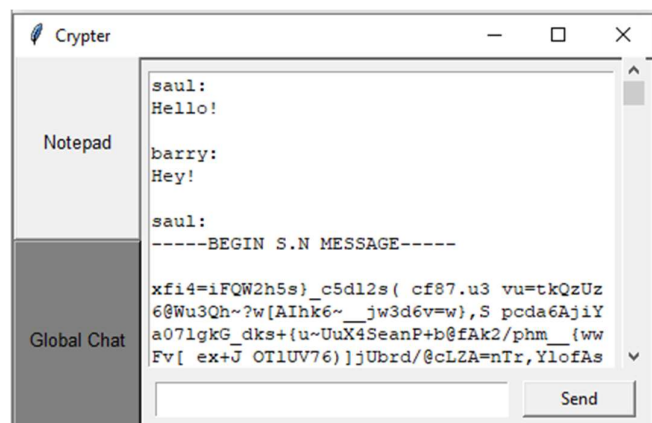
```
chatLog.configure(state='normal')
chatLog.delete('1.0', END)
chatLog.insert(tk.INSERT, chatLines)
chatLog.configure(state='disabled')
```

In order to fix this, I thought of 2 methods. One would be to check if the user is currently on the chat page or the notepad page. If they were on the notepad page, nothing would happen, but if they were on the chat room page, then the program would automatically update the chat widget for them

The other, arguably more simple method I had thought of was to just use a try and except statement to make the program attempt to place the chat into a widget. If this was possible, it would be done and would mean the user is on the chat page. If not, then the program would except and pass, meaning nothing is done and no errors are thrown.

For the reasons of simplicity and also the fact that try and except may help reduce switching pages errors, I went for this method.

```
try:
    #will attempt to place the chat into the widget if the user is on the
global chat page
    chatLog.configure(state='normal')
    chatLog.delete('1.0', END)
    chatLog.insert(tk.INSERT, chatLines)
    chatLog.configure(state='disabled')
except:
    pass
```



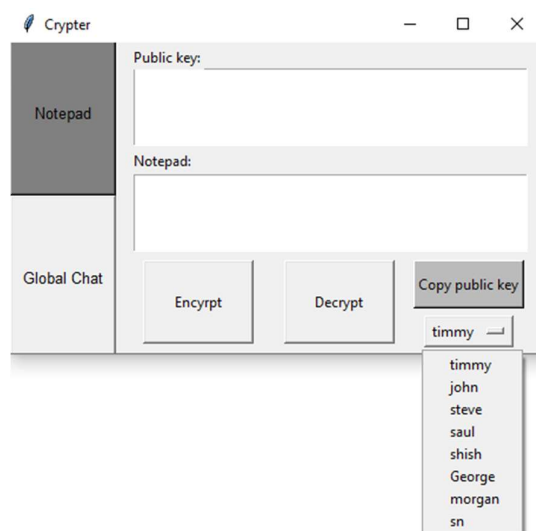
Chat now updated immediately for all connected clients.

Updating of usernames list

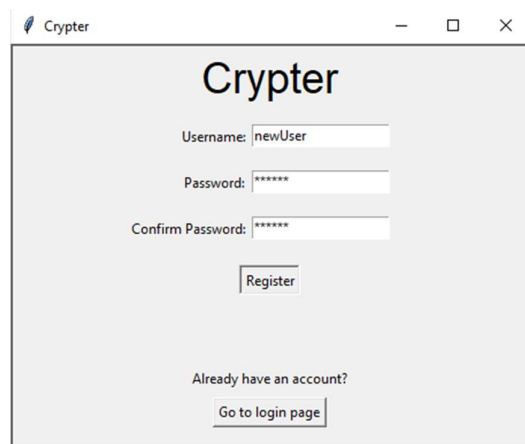
Another issue I noticed during testing with multiple clients at once was the issue of new registering users not showing up in already logged-in users list. This list is needed for choosing who you want to encrypt a message for, so is therefore important to be updated every time a new user joins Crypter.

```
if dataList[0] == 'USERNAMES':
    usernames = dataList.copy()
    usernames.pop(0)
```

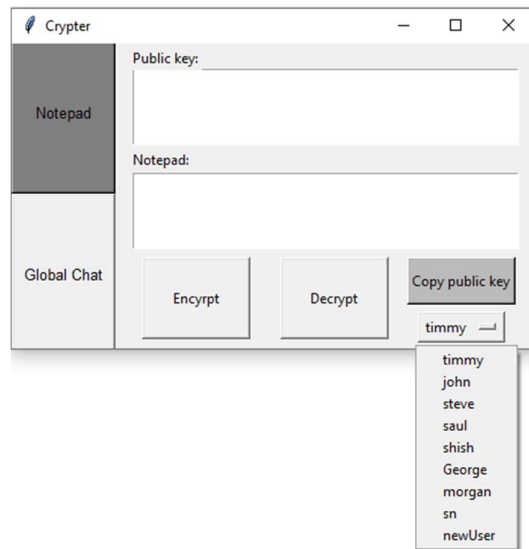
```
message = ['USERNAMES'] + usernames
print(message)
for client in clients:
    client.sendall(bytes(str(message), "utf-8"))
    #sending to all users to update for already logged in users
```



Client 1 already logged in



Client 2 registers account



Username list updates for client 1 while still in same session

Fixing empty passwords

I also noticed during testing that users were once again able to enter empty passwords, which must have happened after I implemented the hashing of passwords. I found that the module I was using to handle the hashing would even hash an empty string, so I changed the code that would check if the password was "", to instead check if the passwords were the specific hashed version of "".

```
emptyPassword =
'e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855'
#'' hashed through sha256
```

```
if username not in usernames and username!='':
    if password1!=password2:
        message = ['Passwords do not match!']
        print(message)
        clients[index].sendall(bytes(str(message), "utf-8"))
```

Handling clients leaving

Finally, the only other error I was receiving when playing around with the program was an shell error appearing when one of the connected clients would leave and close the program. This should not occur however as the server code is still running and therefore the other connected clients sessions should not be affected.

When a client connects to the server, the socket of the client is appended to a list named 'clients'. The idea for managing clients coming and going was to therefore to remove them from the list and shift the index of the clients when someone new joins or leaves. To test this, I had the code within the try and except loop to print out the list of clients and the index of the one leaving.


```

index=clients.index(client)
print(clients)
print(index)
clients.remove(index)
client.close
client= ''

```

```

Connected with ('10.16.53.52', 54740)
Exception in thread Thread-3 (handle):
Traceback (most recent call last):
  File "N:\My Documents\Year 13\Computing\Server\0 server.py", line 111, in handle
    message=client.recv(1024)
ConnectionResetError: [WinError 10054] An existing connection was forcibly closed by the remote host

During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "C:\Program Files\Python310\lib\threading.py", line 1016, in _bootstrap_inner
    self.run()
  File "C:\Program Files\Python310\lib\threading.py", line 953, in run
    self._target(*self._args, **self._kwargs)
  File "N:\My Documents\Year 13\Computing\Server\0 server.py", line 115, in handle
    clients.remove(index)
ValueError: list.remove(x): x not in list

```

As shown by the error, when searching for the number index of the client that needs to be removed, it says that it could not find it in the list of clients. This initially seems like it can't be correct as we have just taken the index in the line above, but I realised after that I instead needed to use `.pop()` rather than `.remove()`. This is because `remove` would take out the first instance of that thing in the list whereas `pop` would remove that specific index of the list, which is what I needed.

```

1
[<socket.socket fd=740, family=AddressFamily.AF_INET,
type=SocketKind.SOCK_STREAM, proto=0, laddr=('10.16.53.55', 21212),
raddr=('10.16.53.53', 58019)>, <socket.socket fd=756,
family=AddressFamily.AF_INET, type=SocketKind.SOCK_STREAM, proto=0,
laddr=('10.16.53.55', 21212), raddr=('10.16.53.52', 55252)>]

0
[<socket.socket fd=740, family=AddressFamily.AF_INET,
type=SocketKind.SOCK_STREAM, proto=0, laddr=('10.16.53.55', 21212),
raddr=('10.16.53.53', 58019)>, <socket.socket fd=756,
family=AddressFamily.AF_INET, type=SocketKind.SOCK_STREAM, proto=0,
laddr=('10.16.53.55', 21212), raddr=('10.16.53.52', 55252)>]

2
[<socket.socket fd=740, family=AddressFamily.AF_INET,
type=SocketKind.SOCK_STREAM, proto=0, laddr=('10.16.53.55', 21212),
raddr=('10.16.53.53', 58019)>, <socket.socket fd=756,
family=AddressFamily.AF_INET, type=SocketKind.SOCK_STREAM, proto=0,
laddr=('10.16.53.55', 21212), raddr=('10.16.53.52', 55252)>, <socket.socket
fd=780, family=AddressFamily.AF_INET, type=SocketKind.SOCK_STREAM, proto=0,
laddr=('10.16.53.55', 21212), raddr=('10.16.53.52', 55285)>]

```

Once I changed to using the `pop` function, the program was now printing out the index of the client that had just left the server and also the complete list of clients. In the above example, the second client to have joined the server (index 1) left first, and then the first client to join (index 0). After this, a third client joined and then left, so as shown the clients list gained another client and then the index 2 is seen leaving.

```
Exception in thread Thread-1 (handle):
Traceback (most recent call last):
  File "N:\My Documents\Year 13\Computing\Server\0 server.py", line 110, in handle
    index=clients.index(client)
ValueError: '' is not in list

During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "C:\Program Files\Python310\lib\threading.py", line 1016, in _bootstrap_inner
    self.run()
  File "C:\Program Files\Python310\lib\threading.py", line 953, in run
    self._target(*self._args, **self._kwargs)
  File "N:\My Documents\Year 13\Computing\Server\0 server.py", line 115, in handle
    index=clients.index(client)
ValueError: '' is not in list
```

Not only was an error received after this for when the final client left, but the list was not updating and removing the current index of the clients that had disconnected. To fix the first problem which was the client being replaced with an empty string (") once they had left, I did a check to make sure the client was an actually client data type and not just an empty string. This ended up fixing both problems in turn and was now removing the clients from the list in the correct order.

```
def handle(client):
    while True:
        try:
            index=clients.index(client)
            message=client.recv(1024)
            receive(message,index)
        except:
            if client!='':
                index=clients.index(client)
                clients.pop(index)
                client.close
                client=''
            print('Client',index,'left the server!')
```

```
0
[<socket.socket fd=876, family=AddressFamily.AF_INET,
type=SocketKind.SOCK_STREAM, proto=0, laddr=('10.16.53.55', 21212),
raddr=('10.16.53.53', 58240)>, <socket.socket fd=824,
family=AddressFamily.AF_INET, type=SocketKind.SOCK_STREAM, proto=0,
laddr=('10.16.53.55', 21212), raddr=('10.16.53.52', 55529)>]
[<socket.socket fd=824, family=AddressFamily.AF_INET,
type=SocketKind.SOCK_STREAM, proto=0, laddr=('10.16.53.55', 21212),
raddr=('10.16.53.52', 55529)>]
0
[<socket.socket fd=824, family=AddressFamily.AF_INET,
type=SocketKind.SOCK_STREAM, proto=0, laddr=('10.16.53.55', 21212),
raddr=('10.16.53.52', 55529)>]
[]
```

```
Server running on IP: 10.16.53.53
Connected with ('10.16.53.55', 50398)
Connected with ('10.16.53.52', 58392)
Client 1 left the server!
Connected with ('10.16.53.52', 58397)
Client 0 left the server!
Connected with ('10.16.53.55', 50415)
Client 0 left the server!
Client 0 left the server!
```

Text visualisation:

[client 1] client 1 joins

c1-0

[client 1, client 2] client 2 joins

c1-0 c2-1

[client 1, client 2, client 3] client 3 joins

c1-0 c2-1 c3-2

[client 1, client 3] client 2 leaves

c1-0 c3-1

[client 1, client 3, client 4] client 4 joins

c1-0 c3-1 c4-2

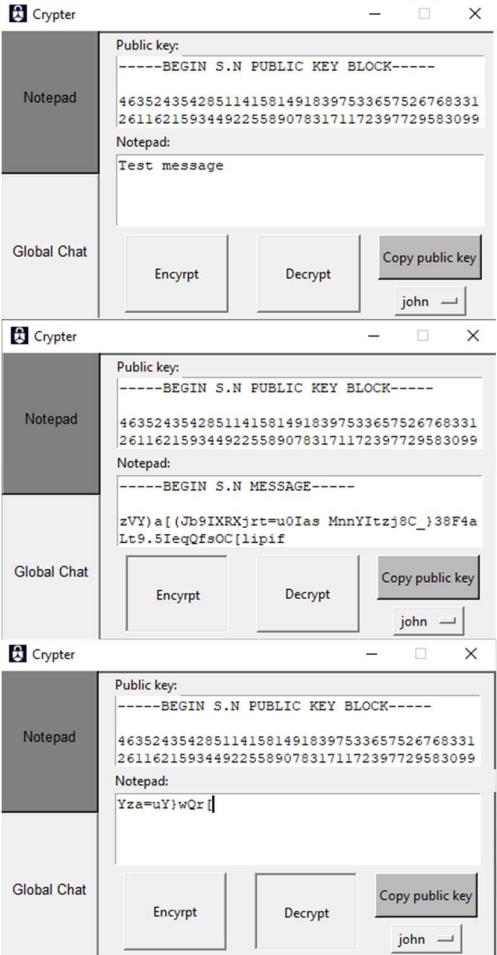
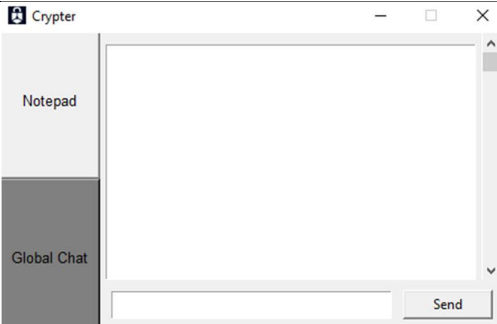
Evaluation

Requirements

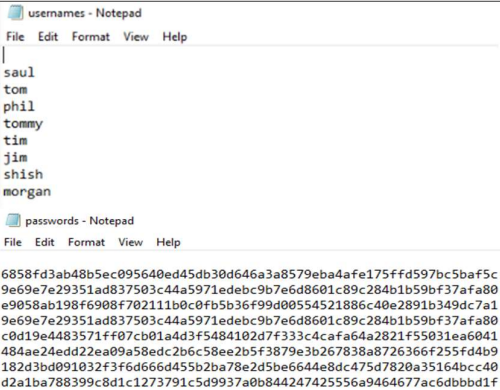
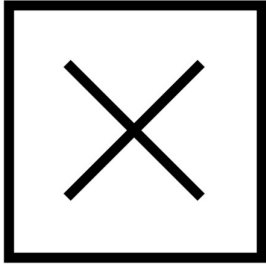
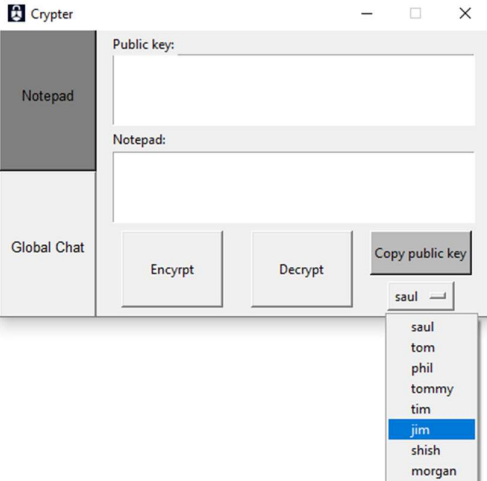
Program requirements from analysis

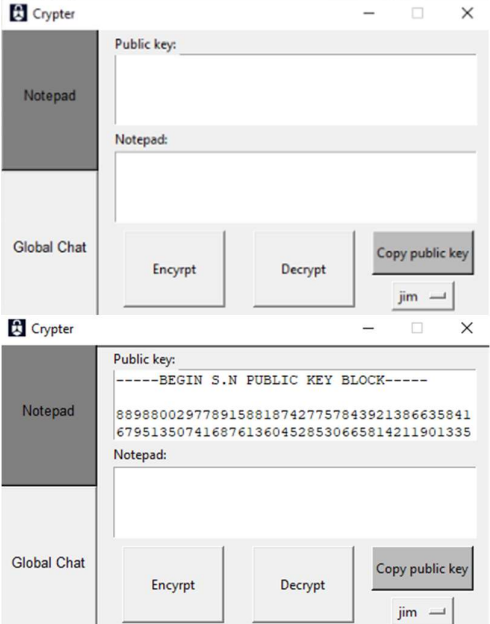
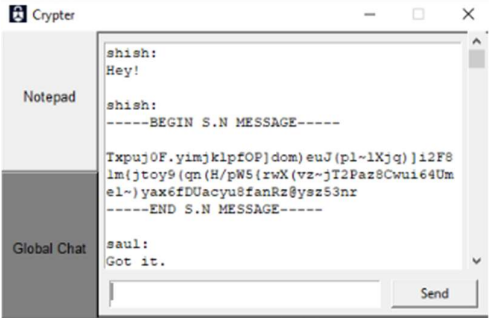
Necessary

Requirement	Completed? (Y/N)		
Online server, for users to connect to		<pre> Server running on IP: 10.16.53.55 Connected with ('10.16.53.53', 53915) Connected with ('10.16.53.52', 53544) </pre>	Throughout testing I found it was completely possible for multiple users to now connect to the server and successfully interact with one another.
A functioning login system			Testing section showed that the login system passed all of my checks, and therefore is fully functional.
A functioning registering system			Testing section showed that the registering system passed all of my checks, and therefore is fully functional.

Encryption and decryption algorithm		The server side encryption / decryption algorithm was another necessary feature of the program, and as shown by the screenshots was fully functional. The screenshots on the left show a user encrypting a message for a different user, and then attempting to decrypt and getting a completely different message back. This shows that the encryption program works.
Key Creation system	<pre>-----BEGIN S.N PUBLIC KEY BLOCK----- 46352435428511415814918397533657526768331 6768331261162159344922558907831711 7239772958309935051719737819803758 2608859081354743383654435330561199 1762889916053158494706340326525908 0953270637301132128555493173344980 7944721856392755743128350198382964 320148153164514435 -----END S.N PUBLIC KEY BLOCK-----</pre>	Key pairs could successfully be generated, and were evidently compatible shown by the working encryption / decryption algorithm.
Clients preferred UI		The clients colour scheme was followed, and the level of simplicity was met according to Joel, so overall I believe the UI had a successful design.

Optional

Requirement	Completed? (Y/N)		
A hashing system for user logins, mainly for passwords			Both the username and password files worked, and the passwords were hashed before being stored.
Extra details for profiles			Unfortunately due to trying to stay within my Gantt chart time constraints, I did not have enough time to add extra details for user accounts to the final version of the program.
Be able to select all users public keys from one account			As shown, I managed to get the users public list working and even fixed the bug where the usernames would not update when a new user registered. A user from here can select a user, request the public key from the server, and then have it copied to clipboard where they can then copy in to the key widget.

			
A chat in which users can publicly send messages			The final optional requirement was the global chat. This also had an issue that I noticed during testing, which was the fact it wouldn't update while a user was on it. I managed to fix this however and also added a feature that would prevent spam.

Client usability

To make sure that my program would work for my client and whoever he is to share it with as it did for me, I would first need to check the re-usability of it. I firstly check what was needed to install in order to run and function the program.

In terms of within the actual to python scripts, a few modules that are not pre-installed with python were used, and therefore will need to be installed for anyone trying to run either the server or the client code.

Server

```
1  #@Saul Needham
2
3  import threading
4  import socket
5  from time import sleep as slp
6  from random import randint as rdi
```

Client

```
1  #@Saul Needham
2
3  import socket
4  import threading
5  import tkinter as tk
6  from tkinter import *
7  from random import randint as rdi
8  from time import sleep as slp
9  from hashlib import sha256
10 import pyperclip
11 import queue
```

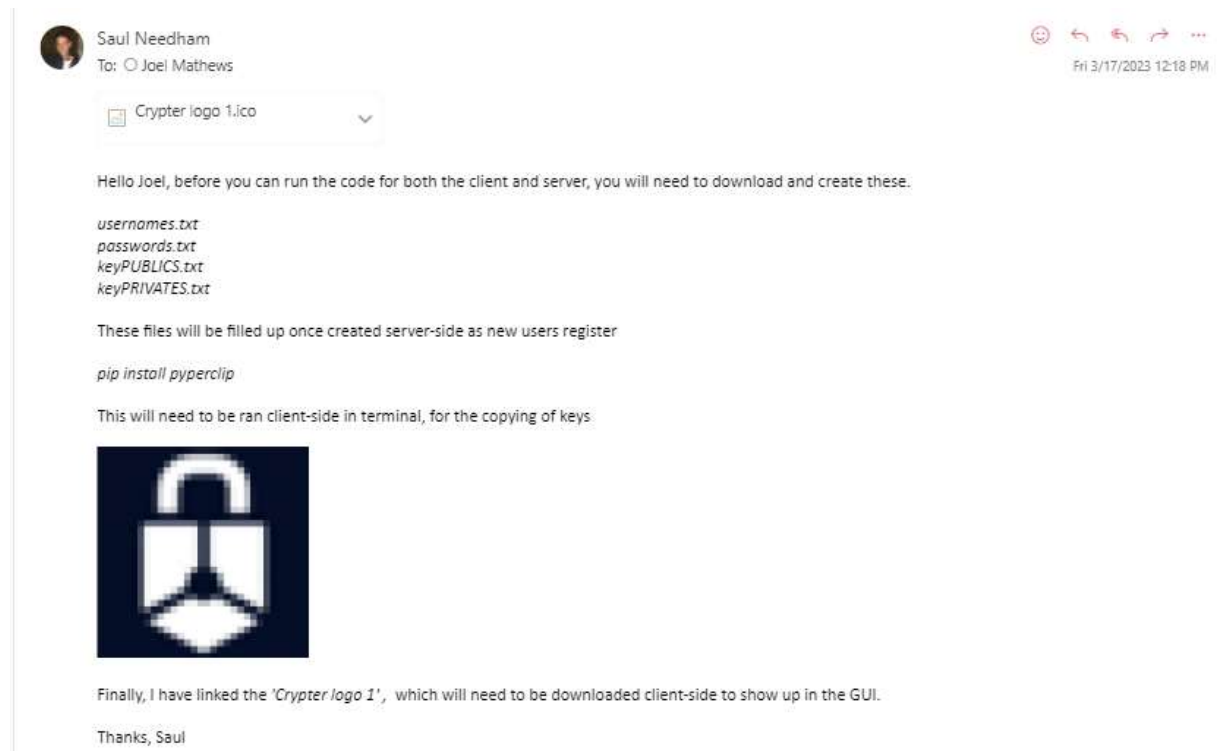
As shown, a few of these libraries are not pre-installed along with python. From a quick check on the python.org website of which modules would need to be additionally installed, I found that my client would need to download between the two programs:

Library	Pre-installed? (Y/N)
socket	
threading	
tkinter	
random	
time	
hashlib	
pyperclip	
queue	

As shown, ‘pyperclip’ would be the only module that would need to be installed to allow the user to run the client script. This module was used so that when a user presses the copy key button, the selected public key is copied to their clipboard, ready to be pasted into the public key widget and used in encryption.

Name	Date modified	Type	Size
0 client	17/03/2023 11:53	Python File	15 KB
0 server	15/03/2023 10:52	Python File	11 KB
Crypter logo 1	03/03/2023 11:59	Icon	5 KB
KEYprivates	15/03/2023 10:25	Text Document	4 KB
KEYpublics	15/03/2023 10:25	Text Document	4 KB
passwords	15/03/2023 10:25	Text Document	1 KB
usernames	15/03/2023 10:25	Text Document	1 KB

Looking through my Crypter folder files, I realised I would also need the client to have the Crypter logo icon file for anyone running the client code, as well as to create 4 files for storing account information for running the server code.



Email detailing necessary program information

Client feedback

Review Response

After sending off the finished program to my client, he provided me with some valuable feedback. Firstly, Joel stated that he was very pleased with the final product, and was happy that all of the necessary features were implemented. He mentioned he was especially happy with the “global chat section of the program and I loved how it updated for all users even when they were currently on the page”. He also mentioned that he was happy with how the encryption and decryption algorithm worked, and felt that it was definitely secure enough for what he was looking for.

He had initially spoken about how he wanted the algorithm to work asymmetrically, using key pairs for each user and was “delighted that the overall program has this feature”. He did however state that the level of asymmetric encryption was not the most secure, as “it did not encrypt with both the receivers public and the senders private, which would have added a level of authentication, and knowing safely who your message is from”. To this, I explained to Joel that the secure hashing system (sha256) made it near impossible to access other users accounts. This made it so whenever a message was received on the global chat, you would always know from the username who it was definitely from.

Overview

Limitations, strengths and improvements

During the development stage of the program, I definitely noticed certain limitations along the way that meant I had to defer from the initial design. One example of this could be the fact that the socket module I was using on python only allowed a maximum of 1024 bytes to be sent between server and client computers and vice-versa. This meant that I had to change the initial planned message character limit to a smaller number to allow any message to be sent.

Another big limitation I had to deal with was the hardware I was using to test the program was not the best for handling several clients at once (hardware specs listed in analysis). This meant that the clients program would have noticeably slower response times from the server once multiple users had left and connected the server. This made final testing fairly slow when carrying out the checks I wanted to do with multiple clients connected, however the code still functioned and passed testing.

One thing that I felt ended up exactly how I wanted was the tkinter GUI, most likely due to spending time in my design stage planning out how I wanted each of the pages to look. My client was also very pleased with the final aesthetics of the user interface, and said it fitted the basic colour scheme and simplistic layout he was originally looking for.

Another thing that I took away from the feedback from Joel was how the encryption method I used could have been improved and made more secure by using both keys to encrypt and decrypt messages, rather than just one for each way. If I was to ever re-attempt or remake the program, I would definitely improve this feature as it would make the whole algorithm a lot more secure.

I would also make an effort to allow more time for development, so that I could fulfil all of my optional requirements and make the program as functional as possible. As shown by my program requirements table, I never actually got round to adding extra details for users accounts. Although not necessary, I and my client Joel believed would have been a valuable addition.

Maintenance for the future

With regards to maintenance of the program, as long as the client is happy with client-side look and functionality code, I would only have to change or update the server side code in order to update Crypter. This means that clients that use Crypter would be able to always use the same code, and only the server would be offline for a short while when the new server is started.

However, if Joel wishes for an update to the client-side code, the code would need to be updated and then re-sent out to everyone who connects to Crypter. As all account information is stored in files securely, this means users credentials and data would remain the same and accounts would not need to be re-created.

Final thoughts

Overall, I am very pleased with how the program turned out, and am glad that I was able to fix all of the bugs and errors I encountered during testing. I think that while my development time was limited and I did not fulfil all optional requirements, I still managed my time fairly well to get the main program working as a whole.

Code

Server code

```
#@Saul Needham

import threading
import socket
from time import sleep as slp
from random import randint as rdi

#-----

clients = []
usernames = []

host = socket.gethostbyname(socket.gethostname())
#gets current IP address of computer
port = 21212 #0 - 65535

server = socket.socket()

server.bind((host, port))
#and runs server on own IP and custom port number
server.listen()
#begins waiting for connections

#-----

KEYBLOCKLEN = 38
MESSAGELEN = 29
#constants that will later be used for key block splitting
keyLen = 10
PGPlen = keyLen+246
#total key length is 256 (246+starter key length of 10)
emptyPassword =
'e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855'
#'' hashed through sha256

fillList=['a', 'b', 'c', 'd', 'e', 'f',
          'g', 'h', 'i', 'j', 'k', 'l',
          'm', 'n', 'o', 'p', 'q', 'r',
          's', 't', 'u', 'v', 'w', 'x',
          'y', 'z', 'A', 'B', 'C', 'D',
          'E', 'F', 'G', 'H', 'I', 'J',
          'K', 'L', 'M', 'N', 'O', 'P',
          'Q', 'R', 'S', 'T', 'U', 'V',
          'W', 'X', 'Y', 'Z', '=', '+',
          ',', '.', '?', '@', '~', '_']
```

```

        '(', ')', '[', ']', '{', '}',
        '/', ' ', '1', '2', '3', '4',
        '5', '6', '7', '8', '9', '0']
#random characters to be placed in encrypted messages
shift="abcdefghijklmnopqrstuvwxyz"
#used for shifting certain charcaters during encryption and decryption

chatLog = ['CHATSEND']
#starting chat log list with the message identifier

#-----

def checkFiles():

    with open('usernames.txt') as file:
        usernames = file.read().split('\n')
        usernames = [x for x in usernames if x]
    with open('passwords.txt') as file:
        passwords = file.read().split('\n')
        passwords = [x for x in passwords if x]
    #opens and splits both usernames and passwords at line breaks and saves to
    lists

    with open('KEYpublics.txt') as file:
        publics = file.read().split('\n\n')
        publics = [x for x in publics if x]
    with open('KEYprivates.txt') as file:
        privates = file.read().split('\n\n')
        privates = [x for x in privates if x]
    #opens and splits both public and private at slashes '/' and saves to
    lists

    return usernames,passwords,publics,privates
    #returns all of the up-to-date users information when the function is run

#

def receive(message,index): #takes the data and index of the client sending
the message

    dataString = message.decode("utf-8")
    dataString = dataString[1:-1].replace("'", "")
    dataList = dataString.split(", ")

    #splits data message sent as a string and turns into list

    if dataList[0] == 'login':
        #checks specific message identifier, and runs respective function

```

```

        loginCheck(dataList,index)
        #both the data and index of client is sent, that currently handling
    elif dataList[0] == 'register':
        registerCheck(dataList,index)
    elif dataList[0] == 'encrypt':
        encrypt(dataList,index)
    elif dataList[0] == 'decrypt':
        decrypt(dataList,index)
    elif dataList[0] == 'keyreq':
        savePublicK(dataList,index)
    elif dataList[0] == 'chatreq':
        sendChat(index)
        #as chat request is a single function and doesn't require anything
        #from the client, just index is sent
    elif dataList[0] == 'send':
        sendMessage(dataList,index)

#

def collect():
    while True:
        client, address = server.accept()
        #server listens for connections and takes clients IP
        print(f'Connected with {str(address)}') #outputs on server
        clients.append(client)
        #and adds client to client list, in order to manage them when leaving

        thread = threading.Thread(target=handle, args=(client,))
        thread.start()
        #begins the handling thread so the server can continuously manage
        #multiple clients

#

def handle(client):
    while True:
        try:
            index=clients.index(client)
            #getting client index from the client list
            message=client.recv(1024) #1024 bytes
            receive(message,index) #runs receive function for client while
            #they're connected
        except:
            if client!='': #makes sure client hasn't already been removed
                index=clients.index(client)
                clients.pop(index)
                client.close
                #closes and removes client that has left from client list

```

```

        client=''
        print('Client',index,'left the server!')

#
def loginCheck(dataList,index):
    usernames,passwords,publics,privates = checkFiles()
    #requests most updated versions of user information files

    username = dataList[1]
    password = dataList[2]
    #username and password entered by client when log in button pressed

    print(dataList)

    if username in usernames and username!='':
        #checks that username is in username list and also isn't empty
        place=usernames.index(username)
        if password==passwords[place]:
            #checks that index of the password is correct for the index of the
            username
            message = ['USERNAMES'] + usernames
            print(message)
            #when succesfully logged in, list of usernames for public key
            copying are sent
            for client in clients:
                client.sendall(bytes(str(message), "utf-8"))
                #sending to all users to update for already logged in users

            message = ['LOGGEDIN',privates[place]]
            print(message)
            clients[index].sendall(bytes(str(message), "utf-8"))
            #message identifier and specific users private is sent to client

        else: #if password isn't correct for username index
            message = ['Incorrect password!']
            print(message)
            clients[index].sendall(bytes(str(message), "utf-8"))
    else: #if username not in the usernames list taken from usernames.txt file
        message = ['Username not registered!']
        print(message)
        clients[index].sendall(bytes(str(message), "utf-8"))

#
def registerCheck(dataList,index):
    print(dataList)
    usernames,passwords,publics,privates = checkFiles()

```

```

#files are again checked to get the most up-to-date usernames list

username = dataList[1]
password1 = dataList[2]
password2 = dataList[3]

if username not in usernames and username.isspace()==False:
#checks if username is not already in the username list and therefore
taken
#also checks the entered username is not blank
    if password1!=password2:
        message = ['Passwords do not match!']
        print(message)
        clients[index].sendall(bytes(str(message), "utf-8"))
    elif password1==emptyPassword or password2==emptyPassword:
#empty password assigned value before ( ' ' through sha256)
        message = ['Password cannot be empty!']
        print(message)
        clients[index].sendall(bytes(str(message), "utf-8"))
    else: #succesful account registering
        publicKtoadd,privateKtoadd,keyID=prodKey()
        #as account has just been created, new key pair is made
        with open('usernames.txt','a') as file:
            file.write('\n'+username)
        with open('passwords.txt','a') as file:
            file.write('\n'+password1)
        with open('KEYpublics.txt','a') as file:
            file.write('\n/\n'+publicKtoadd)
        with open('KEYprivates.txt','a') as file:
            file.write('\n/\n'+privateKtoadd)
        #new account credenitals are added to their respective files

        message = ['REGISTERED'] #client code receives succesful
registration
        print(message)
        clients[index].sendall(bytes(str(message), "utf-8"))

    else: #if username already in usernames list from file
        message = ['Username not available!']
        print(message)
        clients[index].sendall(bytes(str(message), "utf-8"))

#

def prodKey():
#PUBLIC---
    publicKtoadd='-----BEGIN S.N PUBLIC KEY BLOCK-----\n\n'
    for i in range(PGPlen):

```

```

        publicKtoadd+=str(rdi(0,9))
        #makes key by repeated random numbers
        publicKtoadd+='\n-----END S.N PUBLIC KEY BLOCK-----\n'

#PRIVATE---
        privateKtoadd='-----BEGIN S.N PRIVATE KEY BLOCK-----\n\n'
        privateKtoadd+=publicKtoadd[KEYBLOCKLEN:KEYBLOCKLEN+keyLen]
        #takes key ID part of public key and adds to private key
        part=publicKtoadd[(KEYBLOCKLEN+keyLen):(PGPlen+KEYBLOCKLEN)]
        #splits key block and key ID off main body of public key, and adds to
private string
        privateKtoadd+=part[len(part):-1] #reverses key
        privateKtoadd+='\n-----END S.N PRIVATE KEY BLOCK-----\n'

#
        keyID=publicKtoadd[KEYBLOCKLEN:KEYBLOCKLEN+keyLen]
        return publicKtoadd,privateKtoadd,keyID #returns key pair when a new user
registers

def encrypt(dataList,index):
    text = dataList[1]
    PGPC = dataList[2]
    #users message and public key to encrypt with are received by server

    PGPC=PGPC[keyLen:]
    #removing key identifier
    print(text)
    print(PGPC)

    encryptedMessage=''
    encryptBy=0

    if len(text)!=0: #message isn't empty
        if len(text)<=128: #message isn't too long
            if len(PGPC)==246: #valid key entered #256 - keyLen
                for char in text:
                    key=int(PGPC[encryptBy])
                    #using the characters in key to shift text
                    if char in shift:
                        position = shift.find(char)
                        new_position = (position + key) % 26
                        new_character = shift[new_position]
                        encryptedMessage += new_character
                        #simple ceaser shift algorithm that moves along the
charcater

                        #using the 'shift' string abcdef...
                    else:
                        #if character not shiftable

```



```

        encryptedMessage += char
        encryptBy+=1 #moves to next character in message
        ogTextLen=len(encryptedMessage)
        count=0
        for i in range(ogTextLen): #repeats for each character in
original message
            for x in range(int(PGPc[i])):
                #repeats the number currently parsed in the key
                ins=fillList[rdi(0,len(fillList)-1)]
                place=i+x+count
                #takes a random charcater from fillList and counts the
place at which its inserted
                encryptedMessage=encryptedMessage[:place]+ins+encrypte
dMessage[place:]
                count+=int(PGPc[i])
                #increase the place count by the number just repeated from
key

            encryptedMessage+=' '
            encryptedMessage=('-----BEGIN S.N MESSAGE-----\n\n'+
                encryptedMessage+
                '\n-----END S.N MESSAGE-----\n')
            #adds key block around now encrypted message
            message = ['ENCRYPTED',encryptedMessage]
            #sends message identifier and the message to the client who
encrypted

            print(message)
            clients[index].sendall(bytes(str(message), "utf-8"))

        else: #if not 256 character key
            message = ['Invalid public key!']
            print(message)
            clients[index].sendall(bytes(str(message), "utf-8"))

    else:
        message = ['Character limit exceeded!']
        print(message)
        clients[index].sendall(bytes(str(message), "utf-8"))

    else:
        message = ['Message cannot be empty!']
        print(message)
        clients[index].sendall(bytes(str(message), "utf-8"))

#
def decrypt(dataList,index):
    text = dataList[1]
    PGPc = dataList[2]

```

```

print(text)
print(PGPc)

numbersParsed=0
addCheck=0
while addCheck<len(text):
    addCheck+=int(PGPc[numbersParsed])+1
    #sum of all numbers in key length up til it is the same as the length
of the message
    numbersParsed+=1
    #1 also added every iteration for actual character from message
    for x in range(numbersParsed):
        for i in range(int(PGPc[x])):
            text = text[0 : x: ] + text[x+1 : :]
            #removing character from text at position x (getting rid of fill
list)
        decryptedMessage=''
        decryptBy=0
        for char in text:
            key=int(PGPc[decryptBy])
            #parsing through fill list cleared message and taking number from
parsed private key
            if char in shift:
                position = shift.find(char)
                new_position = (position - key) % 26
                new_character = shift[new_position]
                decryptedMessage += new_character
            else:
                decryptedMessage += char
            #shifting backwards along the 'shift' string to reverse ceaser shift
            decryptBy+=1

message = ['DECRYPTED',decryptedMessage]
print(message)
clients[index].sendall(bytes(str(message), "utf-8"))

#

def savePublicK(dataList,index):
    usernames,passwords,publics,privates = checkFiles()
    #files are checked for latest update when a public key is requested
    usernameP = dataList[1]
    place=usernames.index(usernameP)
    #index of user selected is taken
    pubKey = publics[place]
    #and public key of same index is sent back to user from publics list
    message = ['PUBLICKEY',pubKey]

```

```

    print(message)
    clients[index].sendall(bytes(str(message), "utf-8"))

#

def sendChat(index):
    message = chatLog.copy()
    for client in clients:
        client.sendall(bytes(str(message), "utf-8"))
        #sending the updated chat log to all users

#

def sendMessage(dataList,index):
    global chatLog
    username = dataList[1]
    message = (dataList[2])[:-2]
    #getting rid of text widget line break '\n'

    message = message.split('\n')
    messageLines = ''
    for line in message:
        messageLines += line + '\n'
        #replacing line breaks characters from socket with normal line breaks

    print(username+' : '+messageLines)
    chatLog.append(username)
    chatLog.append(messageLines)
    #chatLog list appended to with username of client who sent their message,
and the message
    sendChat(index) #sent to all users

#-----

print(f'Server running on IP: {host}')
collect() #runs main listening loop

```

Client code

```

#@Saul Needham

import socket
import threading
import tkinter as tk
from tkinter import *
from random import randint as rdi
from time import sleep as slp
from hashlib import sha256
import pyperclip
import queue

#-----
--

server = socket.socket()

host = '10.16.53.55' #IP address of server computer
port = 21212 #0 - 65535

server.connect((host, port))

qNotepad = queue.Queue()
qChat = queue.Queue()
qOutput = queue.Queue()

#queues to be used later when inserting text into widgets or onto canvas
#these are necessary as you cannot insert straight from inside the receive
loop

#-----
--

def receive():
    global privateKey, usernames, registered
    while True:
        try:
            message=server.recv(1024) #receiving message from server, not
client
            dataString = message.decode("utf-8")
            dataString = dataString[1:-1].replace("'", "")
            dataList = dataString.split(", ")
            #as socket cannot send lists, having to turn the stringed list
back into a normal list data type

```

```

        normalTerms =
['LOGGEDIN','REGISTERED','USERNAMES','PUBLICKEY','ENCRYPTED','DECRYPTED','CHAT
SEND']

        #all message identifiers that could be sent by the server

        if dataList[0] == 'LOGGEDIN':
            #server will send once the user has a succesful login
            privKey = dataList[1]
            privKey = privKey.split('\n')
            privKeyLines = ''
            for line in privKey:
                privKeyLines += line+'\n'
            #will take the private key from the second index of the
message and remove line breaks
            privKeyLines = privKeyLines[38:-38]
            #keyblocks are then cut off just leaving the numerical part of
the key
            privKeyLines = privKeyLines[(keyLen+1):] #removing key
identifier
            privateKey = privKeyLines[len(privKeyLines):-1]
            #finally reverses the key so that it is compatible with the
users public

        if dataList[0] == 'REGISTERED':
            qOutput.put('Account registered!')
            #as functions cannot be run in the threading function, we have
to queue the message that is to be placed on the GUI
            registered = True

        if dataList[0] == 'USERNAMES':
            usernames = dataList.copy()
            usernames.pop(0)
            #copies the username list and deletes the message identifier
'USERNAMES'

        if dataList[0] == 'PUBLICKEY':
            #received back from server when the user copies a key on the
notepad
            pubKey = dataList[1]
            pubKey = pubKey.split('\n')
            pubKeyLines = ''
            for line in pubKey:
                pubKeyLines += line+'\n'
            pubKeyLines = pubKeyLines[:-2]
            #line breaks are removed and replaced, and the unnecessary
whitespace is cut off
            pyperclip.copy(pubKeyLines)
            #using pyperclip module to copy to clipboard

```

```

        if dataList[0] == 'ENCRYPTED':
            encryptedMessage = dataList[1]
            encryptedMessage = encryptedMessage.split('\n')
            encryptedMessageLines = ''
            for line in encryptedMessage:
                encryptedMessageLines += line+'\n'
            encryptedMessageLines = encryptedMessageLines[:-1]
            qNotepad.put(encryptedMessageLines)
            #encrypted message is recieved from server and then line breaks
are removed
            #message is then put on a queue to be inserted onto GUI, as in
threading loop

        if dataList[0] == 'DECRYPTED':
            decryptedMessage = dataList[1]
            qNotepad.put(decryptedMessage)
            #again message is put on a queue to be inserted onto GUI, as
in threading loop

        if dataList[0] == 'CHATSEND':
            #receiving the chatlog from the server in following form:
            #['CHATSEND',user1,message1,user2,message2,user3,message3...]
            chatHistory = ''
            for i in range(len(dataList)-1):
                if i%2==0: #even index so username
                    chatHistory+=(dataList[i+1]+':\n')
                else: #odd index so message
                    chatHistory+=(dataList[i+1]+' \n')
            #username and messages built onto chatHistory string

            qChat.put(chatHistory)
            #then inserted into the global chat main widget

        if dataList[0] not in normalTerms:
            #single output messages to user
            qOutput.put(dataList[0])
            #these messages appear on the GUI dependant on the page

    except:
        #print('Error occured')
        #client.close()
        server.close()
        break

def queueNotepad():
    while True:
        try:

```

```

        message = qNotepad.get(block=False)
        #attempts to receive a message from the queue
        textEntry.insert(tk.INSERT, message)
        #if successful, the either encrypted or decrypted message is place
in text widget
    except queue.Empty:
        break
        #if no message is sent to the queue, the queue loop breaks
    root.after(50, queueNotepad)
    #resets the queue after 50ms which allows a short wait before resetting

def queueOutput(): #queue used for output messages on GUI
    while True:
        try:
            output = qOutput.get(block=False)
            coverLabel = tk.Label(root, text=' '*128)
            #label of whitespace to cover up previous
            outputLabel = tk.Label(root, text=output)

            try: #attempting to place output message on the three possible
widgets
                #login page in specific blank place
                loginCanvas.create_window(225, 190, window=coverLabel)
                loginCanvas.create_window(225, 190, window=outputLabel)
            except:
                try:
                    #register page in specific blank place
                    registerCanvas.create_window(225, 235, window=coverLabel)
                    registerCanvas.create_window(225, 235, window=outputLabel)
                except:
                    try:
                        #notepad in specific blank place
                        npCanvas.create_window(370, 100, window=coverLabel)
                        npCanvas.create_window(370, 100, window=outputLabel)
                    except:
                        pass

            except queue.Empty:
                break
                #again breaks the queue if no message is received
            root.after(50, queueOutput)
            #and queue is reset

def queueChat():
    while True:
        try:
            chat = qChat.get(block=False)

```

```

        chat = chat.split('\n')
        chatLines = ''
        for line in chat:
            chatLines += line+'\n'
        chatLines = chatLines[:-2]

        #chatLog is placed in chat queue and then line breaks are removed

        try:
            #will attempt to place the chat into the widget if the user is
on the global chat page
            chatLog.configure(state='normal') #makes widget editable
            chatLog.delete('1.0', END)
            chatLog.insert(tk.INSERT, chatLines)
            chatLog.configure(state='disabled') #re-makes widget read only
        except:
            pass

    except queue.Empty:
        break
    root.after(50, queueChat) #resets queue for next time a message is sent

#-----
---

def loginPage():
    global loginCanvas, registerCanvas
    global usernameLogin, passwordLogin
    loginCanvas.destroy()
    registerCanvas.destroy()
    #deleting other pages before creating new login page, so this becomes main
    loginCanvas = tk.Canvas(root, width = 450, height = 350)
    loginCanvas.pack()

    usernameLogin = tk.Entry(loginCanvas)
    usernameLogin.place(x = 210, y = 70, width=120, height=20)
    passwordLogin = tk.Entry(loginCanvas, show="*")
    #makes it so everything typed in password box is replaced by stars '*'
    passwordLogin.place(x = 210, y = 110, width=120, height=20)

    label = tk.Label(root, text='Crypter') #main title label
    label.config(font=('Helvetica bold',28))
    loginCanvas.create_window(225, 30, window=label)
    label = tk.Label(root, text='Username:')
    loginCanvas.create_window(177,80, window=label)
    label = tk.Label(root, text='Password:')
    loginCanvas.create_window(177,120, window=label)
    label = tk.Label(root, text="Don't have an account?")

```



```

loginCanvas.create_window(225,290, window=label)
#adding specific text labels

loginButton = tk.Button(text='Login', command=loginCheck)
loginCanvas.create_window(225, 165, window=loginButton)

switchToLButton = tk.Button(text='Go to register page',
command=registerPage)
loginCanvas.create_window(225, 320, window=switchToLButton)

#two buttons (login and switch page) that will run specific functions from
'command='

#
def registerPage():
    global loginCanvas, registerCanvas
    global usernameRegister, passwordRegister1, passwordRegister2
    loginCanvas.destroy()
    registerCanvas.destroy()
    registerCanvas = tk.Canvas(root, width = 450, height = 350)
    registerCanvas.pack()
    #resetting page when switched to register page

    usernameRegister = tk.Entry(registerCanvas)
    usernameRegister.place(x = 210, y = 70, width=120, height=20)
    passwordRegister1 = tk.Entry(registerCanvas, show="*")
    passwordRegister1.place(x = 210, y = 110, width=120, height=20)
    passwordRegister2 = tk.Entry(registerCanvas, show="*")
    #both pasword widgets replaced with stars '*'
    passwordRegister2.place(x = 210, y = 150, width=120, height=20)

    label = tk.Label(root, text='Crypter')
    label.config(font=('Helvetica bold',28))
    registerCanvas.create_window(225, 30, window=label)
    label = tk.Label(root, text='Username:')
    registerCanvas.create_window(177,80, window=label)
    label = tk.Label(root, text='Password:')
    registerCanvas.create_window(177,120, window=label)
    label = tk.Label(root, text='Confirm Password:')
    registerCanvas.create_window(155,160, window=label)
    label = tk.Label(root, text='Already have an account?')
    registerCanvas.create_window(225,290, window=label)

    loginButton = tk.Button(text='Register', command=registerCheck)
    registerCanvas.create_window(225, 205, window=loginButton)

    switchToLButton = tk.Button(text='Go to login page', command=loginPage)

```

```

registerCanvas.create_window(225, 320, window=switchToLButton)
#buttons again run specific functions from the command parameter

def loginCheck():
    global clientUsername
    password = passwordLogin.get()
    #takes entered password from widget on login page
    hashedPassword = sha256(password.encode('utf-8')).hexdigest()
    #then hashes the password through sha256
    dataList = ['login',
                usernameLogin.get(),
                hashedPassword]

    #data list is sent with message identifier 'login', along with the entered
    username and hashed password
    server.sendall(bytes(str(dataList), "utf-8"))
    slp(1)
    #delay for server to receive and check credentials
    if privateKey!='':
        #privateKey initially starts off blank, but is assigned the users key once
the user logs in
        #this means that the login was succesful if privateKey is no longer blank
        clientUsername = usernameLogin.get()
        loginCanvas.destroy()
        notepadPage()
        #login page is destroyed and replaced by the notepad main UI

def registerCheck():
    password1 = passwordRegister1.get()
    hashedPassword1 = sha256(password1.encode('utf-8')).hexdigest()
    password2 = passwordRegister2.get()
    hashedPassword2 = sha256(password2.encode('utf-8')).hexdigest()
    #both passwords are hashed
    dataList = ['register',
                usernameRegister.get(),
                hashedPassword1,
                hashedPassword2]

    #and data list is sent containing message identifier 'register', username
and passwords
    server.sendall(bytes(str(dataList), "utf-8"))
    slp(1)
    #short wait for server to receive and check credentials
    if registered == True:
        #once succesfully received a register code from server, register is
set to True
        registerCanvas.destroy()
        loginPage()
        #and the user is redirected to the login page

```

```

#-----
-----

def notepadPage():
    global cCanvas, npCanvas
    global textEntry,publicEntry,userVar

    cCanvas.destroy()
    npCanvas.destroy()
    #resets other canvases as user could have switched page by button
    npCanvas = tk.Canvas(root, width = 450, height = 260)
    npCanvas.pack()
    #and re-creates notepad page

    textEntry = tk.Text(npCanvas)
    textEntry.place(x = 105, y = 112, width=335, height=66)
    publicEntry = tk.Text(npCanvas)
    publicEntry.place(x = 105, y = 22, width=335, height=66)
    #adds the public key and notepad text widgets to page

    notepadButton = tk.Button(text='Notepad',width=10,height=8)
    notepadButton.config(font=('Helvetica bold',10), fg='black', bg='grey')
    #changing the colour of the button to grey so the user can see which page
they are on
    npCanvas.create_window(45, 60, window=notepadButton)

    chatButton = tk.Button(text='Global Chat', command=chatPage, width=10,
height=8)
    chatButton.config(font=('Helvetica bold',10), fg='black')
    npCanvas.create_window(45, 200, window=chatButton)

    #buttons to switch between the notepad and chat pages

    encryptButton = tk.Button(text='Encrypt', command=encrypt, width=12,
height=4)
    npCanvas.create_window(160, 220, window=encryptButton)

    decryptButton = tk.Button(text='Decrypt', command=decrypt, width=12,
height=4)
    npCanvas.create_window(280, 220, window=decryptButton)

    copyButton = tk.Button(text='Copy public key', command=savePublicK,
width=12, height=2)
    copyButton.config(bg='#BBBBBB')
    npCanvas.create_window(390, 205, window=copyButton)

    #buttons to encrypt, decrypt and copy a users public key from the list

```

```

    userVar = StringVar(root)
    userVar.set(usernames[0])
    #making the dropdown list of usernames, with the front dropdown being the
first username

    menuUsers= OptionMenu(root, userVar, *usernames)
    #adding the usernames list received from the server to the dropdown
    npCanvas.create_window(390, 245, window=menuUsers)

    labelKey = tk.Label(root, text='Public key:')
    npCanvas.create_window(133, 12, window=labelKey)
    labelNp = tk.Label(root, text='Notepad:')
    npCanvas.create_window(129, 100, window=labelNp)

def chatPage():
    global npCanvas, cCanvas
    global messageEntry, chatLog

    dataList = ['chatreq']
    server.sendall(bytes(str(dataList), "utf-8"))
    #every time the chat page is opened, the most up-to date chat log is
requested

    cCanvas.destroy()
    npCanvas.destroy()
    cCanvas = tk.Canvas(root, width = 450, height = 260)
    cCanvas.pack()
    #replacing switched page to the new global chat page

    notepadButton = tk.Button(text='Notepad',
command=notepadPage,width=10,height=8)
    notepadButton.config(font=('Helvetica bold',10), fg='black')
    cCanvas.create_window(45, 60, window=notepadButton)

    chatButton = tk.Button(text='Global Chat', width=10, height=8)
    chatButton.config(font=('Helvetica bold',10), fg='black', bg='grey')
    #changing the colour of the button to grey so the user can see which page
they are on
    cCanvas.create_window(45, 200, window=chatButton)

    scrollbar = tk.Scrollbar(cCanvas)
    scrollbar.place(x=430, y=0, height=220)
    #making scroll bar for looking through chat log

    chatLog = tk.Text(cCanvas)
    chatLog.place(x=95, y=10, width=330, height=210)
    #making the chat log a text widget, which will later be made read only

```

```

        scrollbar.config(command=chatLog.yview)
        #binding the scrollbar to the chatLog text widget, to scroll vertically
        (yview)

        messageEntry = tk.Text(cCanvas)
        messageEntry.place(x = 100, y = 230, width=250, height=25)

        sendButton = tk.Button(text='Send', command=sendMessage, width=10,
height=1)
        cCanvas.create_window(400, 242, window=sendButton)
        #button that will run function to send entered message

#-----
--

def encrypt():
    coverLabel = tk.Label(root, text=' '*128)
    #label of whitespace to cover up previous message on UI
    npCanvas.create_window(370, 100, window=coverLabel)

    text=(textEntry.get("1.0",tk.END+"-1c")).lower()
    #takes the entered message and makes lower case
    publicKey=publicEntry.get("3.0",tk.END+"-36c")
    dataList = ['encrypt',
                text,
                publicKey]
    #data list containing 'encrypt' identifier, the decrypted message and the
copied public key
    server.sendall(bytes(str(dataList), "utf-8"))
    textEntry.delete('1.0', END)
    #clear the message widget so it can be filled in with message once
encrypted

#

def decrypt():
    coverLabel = tk.Label(root, text=' '*128)
    #label of whitespace
    npCanvas.create_window(370, 100, window=coverLabel)

    text=textEntry.get("3.0",tk.END+"-28c")
    dataList = ['decrypt',
                text,
                privateKey]
    #data list containing 'decrypt' identifier, the encrypted message and
their own private key
    server.sendall(bytes(str(dataList), "utf-8"))

```

```

    textEntry.delete('1.0', END)
    #clears message widget

#

def savePublicK():
    usernameP = userVar.get()
    dataList = ['keyreq',
                usernameP]
    #data list that will request the key of the username they selected from
the list (usernameP)
    server.sendall(bytes(str(dataList), "utf-8"))

#

def sendMessage():
    message = messageEntry.get("1.0",tk.END)
    #takes message currently entered in the message widget
    if message.isspace() == False: #checks if message actually has content
        messageEntry.delete('1.0', END)
        #clears the message entry as your message has been sent, and user will
see appear in global chat
        dataList = ['send',
                    clientUsername,
                    message]
        #send the users name, along with their message to update the chat
        server.sendall(bytes(str(dataList), "utf-8"))

#-----
-----

root = tk.Tk()
root.title('Crypter')

root.iconbitmap("Crypter logo 1.ico")
#binds the logo to the root, to show on all pages of the UI

root.resizable(False, False)
#makes it so the user cannot resize the window, fixing aspect ratio

root.after(50, queueNotepad)
root.after(50, queueChat)
root.after(50, queueOutput)
#starting the queues to allow them to be used

registerCanvas = tk.Canvas(root, width = 450, height = 350)
loginCanvas = tk.Canvas(root, width = 450, height = 350)
cCanvas = tk.Canvas(root, width = 450, height = 350)

```

```
npCanvas = tk.Canvas(root, width = 450, height = 350)
#creates all pages, and these are made global in their respective functions

privateKey='' #set for logging in user
registered = False #set for registering user
keyLen = 10

#-----
-----

receive_thread=threading.Thread(target=receive)
receive_thread.start()
#begins the receive thread to listen for messages from server

loginPage()
#program starts by opening on login canvas

#-----
-----

root.mainloop
#starts tkinter mainloop
```

References

- Front background picture
<https://www.quantamagazine.org/mathematicians-seal-back-door-to-breaking-rsa-encryption-20181217/>
- Top down diagram software
<https://www.lucidchart.com/>
- AI generated Crypter logos
<https://labs.openai.com>
- Pre-installed python libraries
<https://docs.python.org/3/library>